



UNIVERSIDAD
DE GRANADA

GENERACIÓN DE DATOS SINTÉTICOS A PARTIR DE UN
SISTEMA IOT DE DETECCIÓN DE VEHÍCULOS UTILIZANDO
MODELOS TRANSFORMER CON ATENCIÓN EN PYTHON

RICARDO RUIZ FERNÁNDEZ DE ALBA

Trabajo Fin de Grado

Doble Grado en Ingeniería Informática y Matemáticas

Tutores

María Bermúdez Edo

Daniel Bolaños Martínez

FACULTAD DE CIENCIAS

E.T.S. INGENIERÍAS INFORMÁTICA Y DE TELECOMUNICACIÓN

Granada, a 14 de mayo de 2025

ÍNDICE GENERAL

I. BLOQUE I: INTRODUCCIÓN Y PLANTEAMIENTO	6
1. INTRODUCCIÓN Y OBJETIVOS	7
1.1. Introducción	7
1.1.1. Contextualización	7
1.1.2. Descripción del problema	7
1.1.3. Estructura del trabajo	8
1.1.4. Herramientas Matemáticas e Informáticas	8
1.1.5. Bibliografía Fundamental	9
1.2. Objetivos	9
1.2.1. Objetivo General	9
1.2.2. Parte de Matemáticas	9
1.2.3. Parte de Informática	9
II. BLOQUE II: FUNDAMENTOS TEÓRICOS	10
2. FUNDAMENTOS MATEMÁTICOS	11
2.1. Álgebra Lineal y Geometría	11
2.1.1. Vectores y Espacios Vectoriales	11
2.1.2. Matrices y Operaciones Matriciales	12
2.1.3. Tensores y Normas	15
2.2. Análisis Matemático	16
2.2.1. Funciones y Continuidad	17
2.2.2. Cálculo Diferencial Multivariable	17
2.3. Análisis Funcional	20
2.4. Probabilidad y Estadística	22
2.4.1. Estadística Descriptiva	22
2.4.2. Fundamentos de Probabilidad	23
2.4.3. Distribuciones de Probabilidad Discretas Comunes	25
2.5. Funciones Específicas: La Función Softmax	26
2.5.1. Apunte sobre estabilidad numérica	27
2.6. Métodos Numéricos	29
2.6.1. Interpolación lineal	29
2.6.2. Limitaciones de la Interpolación Lineal	30
2.7. Teorema de Aproximación Universal (Cybenko)	31
3. DEL APRENDIZAJE AUTOMÁTICO AL APRENDIZAJE PROFUNDO: FUNDAMENTOS	34
3.1. ¿Qué es el Aprendizaje Automático?	34
3.2. Paradigmas del Aprendizaje Automático	35

3.2.1.	Aprendizaje Supervisado	35
3.2.2.	Aprendizaje No Supervisado	36
3.3.	Evaluación de Modelos y Tratamiento de Datos Faltantes	36
3.3.1.	Métricas de Evaluación	36
3.4.	Modelos Lineales Fundamentales	37
3.4.1.	El Perceptrón	37
3.4.2.	Regresión Lineal	38
3.4.3.	Descenso del Gradiente	38
3.5.	Introducción a los Modelos Transformer	39
3.5.1.	Mecanismo de Atención Escalonada (Scaled Dot-Product At- tention)	39
3.5.2.	Estructura de Capas en un Transformer	42
3.6.	Entrenamiento del Transformer	45
3.7.	Self-Attention-based Imputation for Time Series (SAITS)	47
3.7.1.	Arquitectura del Modelo SAITS	47
3.7.2.	Función de Pérdida para SAITS	48
3.7.3.	Ventajas de Self-Attention para Imputación de Series Temporales	49
3.7.4.	Imputación de Datos Faltantes	50
3.8.	Métricas de Evaluación para Imputación	52
III. BLOQUE III: METODOLOGÍA Y CASO DE ESTUDIO: IMPUTACIÓN DE DA- TOS		53
4.	IMPUTACIÓN DE DATOS DE TRÁFICO Y CONTAMINACIÓN	54
4.1.	Contexto del Problema y Motivación	54
4.2.	Conjuntos de Datos	55
4.2.1.	Dataset de Tráfico General	55
4.2.2.	Dataset de Intersección Tráfico-Contaminación	55
4.3.	Análisis Exploratorio de Datos (EDA)	58
4.3.1.	Análisis de Valores Faltantes	59
4.3.2.	Distribución de Variables	60
4.3.3.	Análisis de Correlaciones	61
4.4.	Metodología: Pipeline de Imputación y Formalización	62
4.4.1.	Desarrollo del Entorno de Imputación	62
4.5.	Preparación de Datos y Configuración Experimental	64
4.5.1.	Carga, Estructura y Preprocesamiento Inicial	64
4.5.2.	División, Escalado y Creación del Escenario de Evaluación	64
4.6.	Problema de Imputación y Objetivo de Evaluación	65
4.7.	Implementación de las Funciones de Imputación	65
4.7.1.	Métodos Baseline o Estadísticos Simples	65
4.7.2.	Métodos Basados en Propagación e Interpolación	66
4.7.3.	Métodos Basados en Modelos Predictivos (Deep Learning)	66
4.8.	Visualización de Series Temporales Imputadas	67
4.9.	Evaluación del Rendimiento	68

IV. BLOQUE IV: CONCLUSIONES Y TRABAJO FUTURO	72
5. CONCLUSIONES Y TRABAJO FUTURO	73
5.1. Trabajo Futuro	73
5.1.1. Mejora de los Modelos Transformer	73
5.1.2. Extensiones a Otros Sistemas IoT	73
5.1.3. Posibles Aplicaciones en Diferentes Dominios	73
Anexos	76
Anexo A: Cronogramas	77
Anexo B: Presupuesto Estimado	79

RESUMEN

Parte I

BLOQUE I: INTRODUCCIÓN Y PLANTEAMIENTO

INTRODUCCIÓN Y OBJETIVOS

1.1 INTRODUCCIÓN

1.1.1 *Contextualización*

La Inteligencia Artificial (IA), impulsada por el avance en algoritmos de aprendizaje automático, ha experimentado un rápido desarrollo debido a su reciente viabilidad de ejecución en hardware y la creciente disponibilidad de datos.

Sus fundamentos matemáticos, incluyendo el análisis, la probabilidad y el álgebra lineal que conforman la teoría del aprendizaje estadístico son esenciales para construir modelos que aprenden de los datos y realizan predicciones.

Como aplicación particular se encuentra la capacidad de estos algoritmos para identificar patrones en datos generados por las tecnologías de Internet de las Cosas (IoT), basadas en redes de sensores y dispositivos conectados a Internet que recopilan y transmiten datos en tiempo real.

1.1.2 *Descripción del problema*

El marco de este trabajo está relacionado con un proyecto de investigación de un sistema IoT implementado en la región de Poqueira, en la Alpujarra granadina ([Universidad de Granada, 2023](#)). El sistema IoT ha procesado la información recogida por los sensores visuales y la ha exportado a un conjunto de datos con múltiples características.

El entrenamiento de modelos de Aprendizaje Automático a partir de estos datos podría predecir variables que mejorasen la gestión de la ocupación de la zona, promoviendo así una mayor sostenibilidad en la comunidad local.

Sin embargo, la complejidad de la información obtenida, la presencia de ruido y especialmente la ausencia de datos, dificultan la tarea. En este contexto, la generación de datos sintéticos se presenta como solución, pues permite añadir al conjunto de

datos original muestras generadas artificialmente que simulan las características de los datos reales. Esta técnica facilita el entrenamiento de modelos más robustos y precisos, mejorando su precisión.

Aunque la literatura existente ha utilizado diversos Modelos de Generación Profunda (DGMs), incluyendo Redes Generativas Antagónicas (GANs) y Modelos de Autoencoders Variacionales (VAEs) (De et al., 2022), en este proyecto nos centraremos en los Modelos Transformers con Atención, pues su capacidad de modelar dependencias a largo plazo en los datos, los presenta como una alternativa prometedora.

1.1.3 Estructura del trabajo

Este trabajo se organiza en cinco capítulos que abordan diferentes aspectos del problema y su solución. La estructura sigue las directrices establecidas para los trabajos de fin de grado.

- **Capítulo 1:** Presenta la Introducción y los Objetivos del proyecto.
- **Capítulo 2:** Enuncia y demuestra los Fundamentos Matemáticos que sustentan el resto del trabajo.
- **Capítulo 3:** Presenta los Fundamentos del Aprendizaje Automático y los modelos clásicos (Perceptrón, Redes Neuronales) necesarios para comprender la relativamente reciente arquitectura del Transformer y su variación SAITS.
- **Capítulo 4:** Describe el desarrollo del experimento, incluyendo detalles sobre el problema, un análisis exploratorio de los datos, su preprocesamiento, las bibliotecas utilizadas para realizar la imputación o generación sintética y las métricas de evaluación asociadas.
- **Capítulo 5:** Presenta las conclusiones y reflexiones finales sobre el trabajo, así como recomendaciones para futuras investigaciones.

1.1.4 Herramientas Matemáticas e Informáticas

Para el desarrollo de este trabajo se utilizarán herramientas matemáticas e informáticas clave. Los fundamentos matemáticos incluyen la base necesaria para comprender la utilización de matrices y tensores, gradiente o métodos numéricos.

La implementación se llevará a cabo en Python, utilizando bibliotecas como *Numpy*, *Pandas*, y *Matplotlib* para el análisis de datos, y *PyPOTS* (Du, 2023) para el entrenamiento de los modelos *Transformers* y *SAITS*.

1.1.5 Bibliografía Fundamental

Para la elaboración del Capítulo 3, destacamos el uso de los libros Learning from Data ([Abu-Mostafa et al., 2012](#)) y Pattern Recognition and Machine Learning ([Bishop, 2006](#))

1.2 OBJETIVOS

1.2.1 Objetivo General

El objetivo general de este trabajo es revisar los fundamentos matemáticos del aprendizaje automático y aplicar este conocimiento en la implementación de modelos generativos, específicamente utilizando arquitecturas Transformer. Diferenciamos dos partes:

1.2.2 Parte de Matemáticas

1. **Establecer los Fundamentos Matemáticos:** Revisar y asimilar los conceptos claves del *Álgebra Lineal*, *Análisis Matemático*, *Probabilidad y Estadística* y *Métodos Numéricos*, que son esenciales para la comprensión y el desarrollo del proyecto.

El objetivo 1.2.2.1 se desarrolla íntegramente en el Capítulo 2 (Chapter 2).

1.2.3 Parte de Informática

1. **Estudiar los Fundamentos del Aprendizaje Automático y Profundo:** Sentar las bases teóricas revisando los conceptos clave del *aprendizaje automático* y *aprendizaje profundo*, así como las métricas de evaluación relevantes.
2. **Definir el Problema y Analizar los Datos:** Centrarse en el problema específico de la imputación de datos faltantes en *series temporales multivariantes* provenientes de un sistema IoT, incluyendo un análisis detallado de las características de dichos datos.
3. **Explorar Metodologías y Realizar Experimentación:** Investigar, describir y contrastar diversas estrategias de imputación (convencionales y basadas en aprendizaje profundo, con énfasis en *SAITS* basado en *Transformers*), y posteriormente, diseñar y ejecutar los experimentos para evaluar su eficacia mediante métricas adecuadas, presentando y analizando comparativamente los resultados.

El objetivo 1.2.3.1 se abordará en el Capítulo 3, y los objetivos 1.2.3.2 y 1.2.3.3 en el Capítulo 4.

Parte II

BLOQUE II: FUNDAMENTOS TEÓRICOS

FUNDAMENTOS MATEMÁTICOS

Este capítulo establece el marco conceptual y las herramientas matemáticas esenciales que servirán de base para comprender los modelos y métodos que presentaremos en los capítulos posteriores. Abordaremos las ramas fundamentales del Álgebra Lineal, el Análisis Matemático, la Probabilidad y Estadística, y los Métodos Numéricos, proporcionando las definiciones y propiedades necesarias de una manera abstracta y general.

2.1 ÁLGEBRA LINEAL Y GEOMETRÍA

Nosotros utilizamos el Álgebra Lineal, un área extensamente tratada en textos como los de [Strang \(2016\)](#) y [Noble and Daniel \(1988\)](#), para representar y manipular datos, así como para describir transformaciones y relaciones en espacios multidimensionales. Estudiamos como objetos centrales los vectores, las matrices y, de forma más general, los tensores, junto con las operaciones que definimos sobre ellos.

2.1.1 Vectores y Espacios Vectoriales

Comenzamos formalizando el concepto de vector, y posteriormente el de matriz, que generaliza la noción de vector y escalar.

Definición 1 (Vector). Definimos un **vector** $\mathbf{v}$ en $\mathbb{K}^n$ (típicamente $\mathbb{R}^n$), donde $\mathbb{K}$ es un cuerpo, como una lista ordenada de n elementos de $\mathbb{K}$ ([Strang, 2016](#)). Lo representamos como una columna de tamaño n :

$$\mathbf{v} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix}$$

o como una fila de tamaño n (vector traspuesto):

$$\mathbf{v}^T = (v_1 \ v_2 \ \dots \ v_n)$$

Llamamos **componentes** del vector a los elementos $v_i \in \mathbb{K}$. El conjunto de todos estos vectores conforma el espacio vectorial $\mathbb{K}^n$.

Sobre los vectores en $\mathbb{K}^n$ definimos la suma y el producto por un escalar componente a componente, operaciones que le confieren la estructura de espacio vectorial (Noble and Daniel, 1988).

EJEMPLO NUMÉRICO EN $\mathbb{R}^3$: Consideremos un vector $\mathbf{u}$ en el espacio tridimensional $\mathbb{R}^3$ (es decir, $n = 3$ y $\mathbb{K} = \mathbb{R}$). Si sus componentes son $u_1 = 2$, $u_2 = -1$, y $u_3 = 5$, entonces escribimos el vector columna como:

$$\mathbf{u} = \begin{pmatrix} 2 \\ -1 \\ 5 \end{pmatrix}$$

Y su vector traspuesto (vector fila) es:

$$\mathbf{u}^T = (2 \ -1 \ 5)$$

2.1.2 Matrices y Operaciones Matriciales

Las matrices son arreglos rectangulares de elementos que nos permiten organizar datos de manera estructurada y representar transformaciones lineales entre espacios vectoriales (Strang, 2016).

Definición 2 (Matriz). Definimos una **matriz** $\mathbf{A}$ de tamaño $m \times n$, con entradas en un cuerpo $\mathbb{K}$ (típicamente $\mathbb{R}$ o $\mathbb{C}$), como una disposición rectangular de m filas y n columnas de elementos de $\mathbb{K}$. Denotamos una matriz por $\mathbf{A} = (a_{ij})$ o $[\mathbf{A}]_{ij} = a_{ij}$, donde $a_{ij} \in \mathbb{K}$ es la entrada en la fila i y la columna j , para $1 \leq i \leq m$ y $1 \leq j \leq n$. Denotamos el conjunto de todas estas matrices como $M_{m \times n}(\mathbb{K})$ o $\mathbb{K}^{m \times n}$ (Bishop, 2006).

Definimos la igualdad entre matrices a partir de la igualdad de sus elementos.

Definición 3 (Igualdad de Matrices). Decimos que dos matrices $\mathbf{A} = (a_{ij})$ y $\mathbf{B} = (b_{ij})$ son **iguales** si pertenecen al mismo espacio $\mathbb{K}^{m \times n}$ (tienen el mismo tamaño) y $a_{ij} = b_{ij}$ para todos $1 \leq i \leq m$, $1 \leq j \leq n$.

Las operaciones básicas que definimos para matrices de igual tamaño son la suma y el producto por un escalar, que realizamos entrada a entrada (Strang, 2016).

Definición 4 (Suma de Matrices). Dadas $\mathbf{A}, \mathbf{B} \in \mathbb{K}^{m \times n}$, definimos su **suma** como la matriz $\mathbf{A} + \mathbf{B} \in \mathbb{K}^{m \times n}$ mediante $[\mathbf{A} + \mathbf{B}]_{ij} = a_{ij} + b_{ij}$.

Definición 5 (Producto por un Escalar). Dado $\mathbf{A} \in \mathbb{K}^{m \times n}$ y un escalar $c \in \mathbb{K}$, definimos el **producto por un escalar** como la matriz $c\mathbf{A} \in \mathbb{K}^{m \times n}$ mediante $[c\mathbf{A}]_{ij} = c \cdot a_{ij}$.

Con estas dos operaciones, el conjunto de matrices de tamaño fijo $\mathbb{K}^{m \times n}$ adquiere la estructura de un espacio vectorial (Noble and Daniel, 1988).

Definición 6 (Espacio Vectorial de Matrices). El conjunto $\mathbb{K}^{m \times n}$, con las operaciones de suma (Definition 4) y producto por un escalar (Definition 5), forma un **espacio vectorial** sobre $\mathbb{K}$. La matriz cero $\mathbf{0}_{m \times n}$ (con todas las entradas 0) actúa como elemento neutro aditivo.

Proposición 1 (Axiomas del Espacio Vectorial $\mathbb{K}^{m \times n}$). Las operaciones definidas en $\mathbb{K}^{m \times n}$ satisfacen los axiomas de espacio vectorial (Strang, 2016).

Demostración. Verificamos que $(\mathbb{K}^{m \times n}, +, \cdot)$ cumple los axiomas de un espacio vectorial sobre $\mathbb{K}$. A modo de ejemplo, demostramos la conmutatividad de la suma. Sean $\mathbf{A}, \mathbf{B} \in \mathbb{K}^{m \times n}$. Queremos demostrar que $\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$. Por la definición de suma de matrices (Definition 4), la entrada (i, j) de $\mathbf{A} + \mathbf{B}$ es:

$$[\mathbf{A} + \mathbf{B}]_{ij} = a_{ij} + b_{ij}$$

La entrada (i, j) de $\mathbf{B} + \mathbf{A}$ es:

$$[\mathbf{B} + \mathbf{A}]_{ij} = b_{ij} + a_{ij}$$

Dado que a_{ij} y b_{ij} son elementos del cuerpo $\mathbb{K}$, y la suma en $\mathbb{K}$ es conmutativa, tenemos que $a_{ij} + b_{ij} = b_{ij} + a_{ij}$ para todo i, j . Por lo tanto, $[\mathbf{A} + \mathbf{B}]_{ij} = [\mathbf{B} + \mathbf{A}]_{ij}$ para todo i, j . Como las matrices tienen el mismo tamaño ($m \times n$) y todas sus entradas correspondientes son iguales, por la definición de igualdad de matrices (Definition 3), concluimos que $\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$.

Verificamos los demás axiomas (asociatividad de la suma, existencia de elemento neutro aditivo - la matriz cero $\mathbf{0}_{m \times n}$, existencia de inverso aditivo $-\mathbf{A}$, asociatividad del producto por escalar, distributividad escalar y vectorial, y elemento neutro multiplicativo $1 \in \mathbb{K}$) de manera similar, apoyándonos en las propiedades correspondientes de la suma y el producto en el cuerpo $\mathbb{K}$ y las definiciones de las operaciones matriciales. La verificación detallada de estos axiomas se puede encontrar en textos de álgebra lineal como el de Strang (2016). $\square$

Además de las operaciones elemento a elemento, el álgebra matricial define un producto que nos permite combinar matrices bajo ciertas condiciones de compatibilidad de tamaño.

Definición 7 (Producto Matricial). Definimos el **producto matricial** de $\mathbf{A} \in \mathbb{K}^{m \times p}$ y $\mathbf{B} \in \mathbb{K}^{p \times n}$ como la matriz $\mathbf{C} = \mathbf{AB} \in \mathbb{K}^{m \times n}$, mediante:

$$[\mathbf{AB}]_{ij} = c_{ij} = \sum_{k=1}^p a_{ik}b_{kj}$$

para $1 \leq i \leq m$ y $1 \leq j \leq n$ (Bishop, 2006). *Nota:* Definimos el producto $\mathbf{AB}$ solo si el número de columnas de $\mathbf{A}$ es igual al número de filas de $\mathbf{B}$.

El producto matricial posee propiedades importantes, aunque no es conmutativo en general (Strang, 2016).

Proposición 2 (Propiedades del Producto Matricial). Sean $\mathbf{A}, \mathbf{B}, \mathbf{C}$ matrices de tamaños compatibles para que las operaciones estén definidas, y c un escalar. Nosotros establecemos las siguientes propiedades (Noble and Daniel, 1988):

1. Asociatividad: $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$
2. Distributividad: $\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}$ y $(\mathbf{A} + \mathbf{B})\mathbf{C} = \mathbf{AC} + \mathbf{BC}$
3. Compatibilidad escalar: $c(\mathbf{AB}) = (c\mathbf{A})\mathbf{B} = \mathbf{A}(c\mathbf{B})$
4. No Conmutatividad general: $\mathbf{AB} \neq \mathbf{BA}$ en general.

Además del producto matricial estándar, a menudo resulta útil realizar operaciones elemento a elemento.

Definición 8 (Producto Elemento a Elemento - Hadamard). Dadas $\mathbf{A}, \mathbf{B} \in \mathbb{K}^{m \times n}$ (mismo tamaño), definimos su **producto elemento a elemento** o **producto de Hadamard** como la matriz $\mathbf{A} \odot \mathbf{B} \in \mathbb{K}^{m \times n}$ mediante:

$$[\mathbf{A} \odot \mathbf{B}]_{ij} = a_{ij} \cdot b_{ij}$$

Este producto es mencionado en contextos de aprendizaje profundo (Goodfellow et al., 2016).

Otra operación fundamental es la transposición, que intercambia filas por columnas.

Definición 9 (Matriz Traspuesta). Dada $\mathbf{A} \in \mathbb{K}^{m \times n}$, definimos su **matriz traspuesta**, $\mathbf{A}^T \in \mathbb{K}^{n \times m}$, mediante:

$$[\mathbf{A}^T]_{ij} = a_{ji}$$

(Strang, 2016).

La transposición tiene propiedades que interactúan con las operaciones matriciales.

Proposición 3 (Propiedades de la Traspuesta). Sean $\mathbf{A}, \mathbf{B}$ matrices de tamaños compatibles para que las operaciones estén definidas, y c un escalar. Nosotros enunciamos las siguientes propiedades (Bishop, 2006):

1. $(\mathbf{A}^T)^T = \mathbf{A}$

2. $(\mathbf{A} + \mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T$
3. $(c\mathbf{A})^T = c\mathbf{A}^T$
4. $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$

2.1.3 Tensores y Normas

Expandiendo la noción de matrices, los tensores nos permiten representar datos y operaciones en dimensiones arbitrarias, un concepto fundamental en el aprendizaje automático moderno. Para un tratamiento más formal sobre tensores y sus descomposiciones, se puede consultar a [Kolda and Bader \(2009\)](#).

Definición 10 (Tensor y Orden). Un **tensor** $\mathbf{A}$ de **orden** k (o grado k) sobre el campo de los números reales $\mathbb{R}$ es una generalización de escalares ($k = 0$), vectores ($k = 1$) y matrices ($k = 2$). Lo podemos representar como un array multidimensional cuyos elementos direccionamos usando k índices. Si las dimensiones a lo largo de cada índice son $d_1, d_2, \dots, d_k$, entonces el tensor pertenece al espacio $\mathbb{R}^{d_1 \times d_2 \times \dots \times d_k}$. Denotamos un elemento del tensor $\mathbf{A}$ usando la notación de índices como $[\mathbf{A}]_{i_1 i_2 \dots i_k}$, donde cada índice i_j varía sobre la extensión de la j -ésima dimensión (i.e., $1 \leq i_j \leq d_j$) ([Goodfellow et al., 2016](#)).

Extendemos de manera natural a los tensores las operaciones elemento a elemento definidas para matrices.

Definición 11 (Operaciones Elemento a Elemento para Tensores). Sean $\mathbf{A}, \mathbf{B}$ dos tensores del mismo orden k y con idénticas dimensiones $d_1 \times \dots \times d_k$. Sea $c \in \mathbb{R}$ un escalar y $f : \mathbb{R} \rightarrow \mathbb{R}$ una función escalar. Definimos ([Goodfellow et al., 2016](#)):

- **Suma/Resta:** $\mathbf{C} = \mathbf{A} \pm \mathbf{B}$ con $[\mathbf{C}]_{i_1 \dots i_k} = [\mathbf{A}]_{i_1 \dots i_k} \pm [\mathbf{B}]_{i_1 \dots i_k}$.
- **Multipliación por Escalar:** $\mathbf{C} = c\mathbf{A}$ con $[\mathbf{C}]_{i_1 \dots i_k} = c \cdot [\mathbf{A}]_{i_1 \dots i_k}$.
- **Producto de Hadamard:** $\mathbf{C} = \mathbf{A} \odot \mathbf{B}$ con $[\mathbf{C}]_{i_1 \dots i_k} = [\mathbf{A}]_{i_1 \dots i_k} \cdot [\mathbf{B}]_{i_1 \dots i_k}$ (Definition 8 generalizada).
- **Aplicación de Función Escalar:** $f(\mathbf{A})$ con $[f(\mathbf{A})]_{i_1 \dots i_k} = f([\mathbf{A}]_{i_1 \dots i_k})$.

Para medir la "magnitud" de un tensor o de los errores entre tensores, utilizamos el concepto de norma ([Goodfellow et al., 2016](#)).

Definición 12 (Norma L_p de un Tensor). Definimos la **norma** L_p de un tensor $\mathbf{A} \in \mathbb{R}^{d_1 \times \dots \times d_k}$, para $1 \leq p < \infty$, como:

$$\|\mathbf{A}\|_p = \left(\sum_{i_1=1}^{d_1} \dots \sum_{i_k=1}^{d_k} |[\mathbf{A}]_{i_1 \dots i_k}|^p \right)^{1/p}$$

donde $[\mathbf{A}]_{i_1 \dots i_k}$ denota el elemento del tensor $\mathbf{A}$ correspondiente a los índices $i_1, \dots, i_k$. Para $p = \infty$, la norma L_∞ (también conocida como norma del máximo) es el máximo valor absoluto de sus elementos:

$$\|\mathbf{A}\|_\infty = \max_{i_1, \dots, i_k} |[\mathbf{A}]_{i_1 \dots i_k}|$$

Casos particulares importantes son la **norma** L_1 ($p = 1$), que es la suma de los valores absolutos de todos los elementos, y la **norma** L_2 ($p = 2$). Para matrices (tensores de orden $k = 2$), la norma L_2 es comúnmente conocida como la **norma de Frobenius** (Bishop, 2006; Golub and Van Loan, 2013).

La norma L_1 suma los valores absolutos de los elementos. La norma L_2 (o norma de Frobenius para matrices) está intrínsecamente relacionada con el producto escalar. Si consideramos el tensor $\mathbf{A}$ como un vector aplanado $\text{vec}(\mathbf{A})$ en un espacio de dimensión $d_1 \times \dots \times d_k$, entonces la norma L_2 al cuadrado es el producto escalar de este vector consigo mismo:

$$\|\mathbf{A}\|_2^2 = \sum_{i_1=1}^{d_1} \dots \sum_{i_k=1}^{d_k} ([\mathbf{A}]_{i_1 \dots i_k})^2 = \langle \text{vec}(\mathbf{A}), \text{vec}(\mathbf{A}) \rangle$$

Para una matriz $\mathbf{M} \in \mathbb{R}^{m \times n}$, la norma de Frobenius al cuadrado también puede expresarse como la traza del producto de la matriz por su traspuesta (Golub and Van Loan, 2013):

$$\|\mathbf{M}\|_F^2 = \|\mathbf{M}\|_2^2 = \sum_{i=1}^m \sum_{j=1}^n ([\mathbf{M}]_{ij})^2 = \text{Tr}(\mathbf{M}^T \mathbf{M}) = \text{Tr}(\mathbf{M} \mathbf{M}^T)$$

Ambas normas, L_1 y L_2 , son cruciales cuando definimos métricas de error y funciones de regularización en aprendizaje automático (Bishop, 2006).

EJEMPLO DE CÁLCULO DE LA NORMA L_2 (NORMA DE FROBENIUS): Consideremos la siguiente matriz $\mathbf{B} \in \mathbb{R}^{2 \times 3}$:

$$\mathbf{B} = \begin{pmatrix} 1 & -2 & 0 \\ 3 & 4 & -1 \end{pmatrix}$$

Calculamos la norma L_2 (o norma de Frobenius) de $\mathbf{B}$ como:

$$\begin{aligned} \|\mathbf{B}\|_2 &= \left(\sum_{i=1}^2 \sum_{j=1}^3 |[\mathbf{B}]_{ij}|^2 \right)^{1/2} \\ &= \sqrt{1^2 + (-2)^2 + 0^2 + 3^2 + 4^2 + (-1)^2} \\ &= \sqrt{1 + 4 + 0 + 9 + 16 + 1} \\ &= \sqrt{31} \end{aligned}$$

2.2 ANÁLISIS MATEMÁTICO

El Análisis Matemático, particularmente el cálculo diferencial, nos proporciona herramientas para estudiar la variación y el cambio, lo cual es esencial para comprender cómo se comportan las funciones y cómo optimizarlas. Textos clásicos como los de [Rudin \(1976\)](#) y [Apostol \(1974\)](#) cubren estos temas en profundidad.

2.2.1 Funciones y Continuidad

Comenzamos con el concepto fundamental de función, una regla que asigna a cada elemento de un conjunto de partida un único elemento de un conjunto de llegada.

Definición 13 (Función). Una **función** f de un conjunto A a un conjunto B , que denotamos $f : A \rightarrow B$, es una regla que asigna a cada elemento $x \in A$ un único elemento $f(x) \in B$. Llamamos a A el **dominio** de f , y a B el **codominio**. La **imagen** de f , denotada $\text{Im}(f)$, es el conjunto $\{f(x) | x \in A\} \subseteq B$ ([Apostol, 1974](#)).

En el contexto que nos ocupa, trabajaremos principalmente con funciones cuyo dominio y codominio son subconjuntos de espacios euclídeos $\mathbb{R}^n$ o $\mathbb{R}^m$. La continuidad, un concepto clave, describe funciones cuyas salidas varían de manera suave con pequeñas variaciones en sus entradas ([Rudin, 1976](#)).

Definición 14 (Continuidad). Decimos que una función $f : A \subseteq \mathbb{R}^n \rightarrow \mathbb{R}^m$ es **continua** en un punto $\mathbf{a} \in A$ si para todo $\epsilon > 0$, existe un $\delta > 0$ tal que si $\mathbf{x} \in A$ y $\|\mathbf{x} - \mathbf{a}\|_2 < \delta$, entonces $\|f(\mathbf{x}) - f(\mathbf{a})\|_2 < \epsilon$. Si f es continua en todos los puntos de A , decimos que f es continua en A . Esta es la definición $\epsilon - \delta$ estándar de continuidad ([Apostol, 1974](#); [Rudin, 1976](#)).

2.2.2 Cálculo Diferencial Multivariable

Extendemos la noción de derivada a funciones de varias variables o funciones con valores vectoriales, siguiendo el tratamiento de autores como [Apostol \(1974\)](#).

Definición 15 (Derivada Parcial). Sea $f : U \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ definida en un conjunto abierto U . Definimos la **derivada parcial** de f respecto a la i -ésima variable x_i en un punto $\mathbf{a} \in U$ como el límite:

$$\frac{\partial f}{\partial x_i}(\mathbf{a}) = \lim_{h \rightarrow 0} \frac{f(\mathbf{a} + h\mathbf{e}_i) - f(\mathbf{a})}{h}$$

si el límite existe, donde $\mathbf{e}_i$ es el i -ésimo vector canónico de la base estándar de $\mathbb{R}^n$ ([Rudin, 1976](#)).

Es importante destacar que la mera existencia de todas las derivadas parciales de una función f en un punto $\mathbf{a}$ no garantiza que f sea diferenciable en $\mathbf{a}$. La diferenciabilidad es una condición más fuerte que implica que la función puede ser bien aproximada localmente por una transformación lineal (su diferencial) (Apostol, 1974; Rudin, 1976).

Una condición suficiente común para la diferenciabilidad es que la función sea de **clase** C^1 . Decimos que una función f es de clase C^1 en un conjunto abierto U si todas sus derivadas parciales $\frac{\partial f}{\partial x_i}$ existen en U y son, además, funciones continuas en U . Si las derivadas parciales de f existen en un entorno de $\mathbf{a}$ y son continuas en $\mathbf{a}$ mismo, entonces f es diferenciable en $\mathbf{a}$ (Apostol, 1974).

CONTRA EJEMPLO: Consideremos la función $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ definida por:

$$f(x, y) = \begin{cases} \frac{xy}{x^2+y^2} & \text{si } (x, y) \neq (0, 0) \\ 0 & \text{si } (x, y) = (0, 0) \end{cases}$$

Calculamos las derivadas parciales en el origen $(0, 0)$:

$$\begin{aligned} \frac{\partial f}{\partial x}(0, 0) &= \lim_{h \rightarrow 0} \frac{f(h, 0) - f(0, 0)}{h} = \lim_{h \rightarrow 0} \frac{(h \cdot 0)/(h^2 + 0^2) - 0}{h} = \lim_{h \rightarrow 0} \frac{0}{h} = 0 \\ \frac{\partial f}{\partial y}(0, 0) &= \lim_{h \rightarrow 0} \frac{f(0, h) - f(0, 0)}{h} = \lim_{h \rightarrow 0} \frac{(0 \cdot h)/(0^2 + h^2) - 0}{h} = \lim_{h \rightarrow 0} \frac{0}{h} = 0 \end{aligned}$$

Ambas derivadas parciales existen en $(0, 0)$ y son iguales a cero. Sin embargo, la función f no es continua en $(0, 0)$, como se discute en Apostol (1974) o Rudin (1976) para ejemplos similares. Si nos acercamos al origen a lo largo de la recta $y = mx$, obtenemos:

$$\lim_{x \rightarrow 0} f(x, mx) = \lim_{x \rightarrow 0} \frac{x(mx)}{x^2 + (mx)^2} = \lim_{x \rightarrow 0} \frac{mx^2}{x^2(1 + m^2)} = \frac{m}{1 + m^2}$$

Dado que este límite depende de la pendiente m de la trayectoria de aproximación, el límite general $\lim_{(x,y) \rightarrow (0,0)} f(x, y)$ no existe. Como la función no es continua en $(0, 0)$, no puede ser diferenciable en $(0, 0)$, a pesar de que todas sus derivadas parciales existan en ese punto.

Para funciones de $\mathbb{R}^n$ a $\mathbb{R}$, podemos agrupar todas las derivadas parciales en un vector llamado gradiente.

Definición 16 (Gradiente). Sea $f : U \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ una función real-valorada tal que todas sus derivadas parciales existen en un punto $\mathbf{a} \in U$. Definimos el **vector gradiente** de f en $\mathbf{a}$, denotado $\nabla f(\mathbf{a})$, como el vector en $\mathbb{R}^n$ cuyas componentes son las derivadas parciales de f en $\mathbf{a}$:

$$\nabla f(\mathbf{a}) = \left(\frac{\partial f}{\partial x_1}(\mathbf{a}), \frac{\partial f}{\partial x_2}(\mathbf{a}), \dots, \frac{\partial f}{\partial x_n}(\mathbf{a}) \right)^T$$

Si f es diferenciable en $\mathbf{a}$, el gradiente $\nabla f(\mathbf{a})$ indica la dirección y magnitud del mayor aumento local de f en el punto $\mathbf{a}$ (Apostol, 1974; Bishop, 2006).

Cuando el codominio de la función es vectorial ($\mathbb{R}^m$), organizamos las derivadas parciales en una matriz.

Proposición 4 (Diferenciabilidad y Gradiente para Campos Escalares). *Para un campo escalar $f : \Omega \rightarrow \mathbb{R}$ donde Ω es un abierto de $\mathbb{R}^N$, y un punto $a \in \Omega$, las siguientes afirmaciones son equivalentes (Apostol, 1974):*

1. f es diferenciable en a .
2. f es parcialmente derivable en a (es decir, existe $\nabla f(a)$) y verificamos que:

$$\lim_{\mathbf{x} \rightarrow \mathbf{a}} \frac{f(\mathbf{x}) - f(\mathbf{a}) - \langle \nabla f(\mathbf{a}), \mathbf{x} - \mathbf{a} \rangle}{\|\mathbf{x} - \mathbf{a}\|_2} = 0 \quad (1)$$

donde $\langle \cdot, \cdot \rangle$ denota el producto escalar usual en $\mathbb{R}^N$.

En caso de que se cumplan (i) y (ii), tenemos que la diferencial de f en a aplicada a un vector $v \in \mathbb{R}^N$, denotada $Df(a)v$, es:

$$Df(a)v = \langle \nabla f(a), v \rangle \quad (\text{también escrito como } \nabla f(a) \cdot v \text{ o } (\nabla f(a))^T v) \quad (2)$$

Definición 17 (Matriz Jacobiana). Sea $\mathbf{f} : U \subseteq \mathbb{R}^n \rightarrow \mathbb{R}^m$ una función vectorial $\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), \dots, f_m(\mathbf{x}))^T$, donde cada $f_i : U \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$. Si todas las derivadas parciales $\frac{\partial f_i}{\partial x_j}(\mathbf{x})$ existen en un punto $\mathbf{x} \in U$, definimos la **matriz Jacobiana** de $\mathbf{f}$ en $\mathbf{x}$, denotada $\mathbf{J}_{\mathbf{f}}(\mathbf{x})$, como la matriz de tamaño $m \times n$ cuyas entradas son:

$$[\mathbf{J}_{\mathbf{f}}(\mathbf{x})]_{ij} = \frac{\partial f_i}{\partial x_j}(\mathbf{x})$$

Notar que si $m = 1$, entonces $\mathbf{f}$ es una función real-valorada f , y la matriz Jacobiana es el vector gradiente traspuesto: $\mathbf{J}_f(\mathbf{x}) = (\nabla f(\mathbf{x}))^T$ (Apostol, 1974; Bishop, 2006).

Una regla fundamental para derivar funciones compuestas es la regla de la cadena (Apostol, 1974; Rudin, 1976).

Teorema 1 (Regla de la Cadena Multivariable). Sean $U \subseteq \mathbb{R}^n$, $V \subseteq \mathbb{R}^m$ conjuntos abiertos. Sean $\mathbf{g} : U \rightarrow V$ diferenciable en $\mathbf{x}_0 \in U$, y $\mathbf{f} : V \rightarrow \mathbb{R}^p$ diferenciable en $\mathbf{y}_0 = \mathbf{g}(\mathbf{x}_0) \in V$. La composición $\mathbf{h} = \mathbf{f} \circ \mathbf{g} : U \rightarrow \mathbb{R}^p$ es diferenciable en $\mathbf{x}_0$, y su matriz Jacobiana en $\mathbf{x}_0$ es el producto de las matrices Jacobianas de $\mathbf{f}$ y $\mathbf{g}$:

$$\mathbf{J}_{\mathbf{h}}(\mathbf{x}_0) = \mathbf{J}_{\mathbf{f}}(\mathbf{y}_0)\mathbf{J}_{\mathbf{g}}(\mathbf{x}_0)$$

(Apostol, 1974).

Demostración. Sea $\mathbf{h}(\mathbf{x}) = \mathbf{f}(\mathbf{g}(\mathbf{x}))$. Escribimos las funciones componentes: $\mathbf{x} = (x_1, \dots, x_n)^T$, $\mathbf{y} = \mathbf{g}(\mathbf{x}) = (g_1(\mathbf{x}), \dots, g_m(\mathbf{x}))^T$, y $\mathbf{z} = \mathbf{h}(\mathbf{x}) = \mathbf{f}(\mathbf{y}) = (f_1(\mathbf{y}), \dots, f_p(\mathbf{y}))^T$. Queremos encontrar la entrada (k, j) de la matriz Jacobiana $\mathbf{J}_{\mathbf{h}}(\mathbf{x}_0)$, que por definición es $\frac{\partial h_k}{\partial x_j}(\mathbf{x}_0)$. La función h_k es $f_k(g_1(\mathbf{x}), \dots, g_m(\mathbf{x}))$. Aplicando la regla de la cadena del cálculo multivariable para funciones escalares, como se detalla en [Apostol \(1974\)](#), calculamos la derivada parcial de h_k con respecto a x_j como:

$$\frac{\partial h_k}{\partial x_j}(\mathbf{x}_0) = \sum_{l=1}^m \frac{\partial f_k}{\partial y_l}(\mathbf{y}_0) \frac{\partial g_l}{\partial x_j}(\mathbf{x}_0)$$

donde $\mathbf{y}_0 = \mathbf{g}(\mathbf{x}_0)$. Examinamos el lado derecho de la ecuación. El término $\frac{\partial f_k}{\partial y_l}(\mathbf{y}_0)$ es la entrada (k, l) de la matriz Jacobiana $\mathbf{J}_{\mathbf{f}}(\mathbf{y}_0)$. El término $\frac{\partial g_l}{\partial x_j}(\mathbf{x}_0)$ es la entrada (l, j) de la matriz Jacobiana $\mathbf{J}_{\mathbf{g}}(\mathbf{x}_0)$. La suma $\sum_{l=1}^m [\mathbf{J}_{\mathbf{f}}(\mathbf{y}_0)]_{kl} [\mathbf{J}_{\mathbf{g}}(\mathbf{x}_0)]_{lj}$ es precisamente la definición de la entrada (k, j) del producto matricial $\mathbf{J}_{\mathbf{f}}(\mathbf{y}_0) \mathbf{J}_{\mathbf{g}}(\mathbf{x}_0)$. Por lo tanto, hemos demostrado que:

$$[\mathbf{J}_{\mathbf{h}}(\mathbf{x}_0)]_{kj} = \frac{\partial h_k}{\partial x_j}(\mathbf{x}_0) = [\mathbf{J}_{\mathbf{f}}(\mathbf{y}_0) \mathbf{J}_{\mathbf{g}}(\mathbf{x}_0)]_{kj}$$

Como esto es válido para todo $1 \leq k \leq p$ y $1 \leq j \leq n$, las matrices son iguales: $\mathbf{J}_{\mathbf{h}}(\mathbf{x}_0) = \mathbf{J}_{\mathbf{f}}(\mathbf{y}_0) \mathbf{J}_{\mathbf{g}}(\mathbf{x}_0)$. (Nota: Esta prueba asume la validez de la regla de la cadena para derivar una función escalar f_k compuesta con una función vectorial $\mathbf{g}$. Una prueba más rigurosa se basa en la definición de diferenciabilidad mediante aproximaciones lineales y el manejo cuidadoso de los términos de error, como se encuentra en [Apostol \(1974\)](#) o [Rudin \(1976\)](#)). $\square$

Esta regla es crucial para el cálculo de gradientes en funciones compuestas, una operación recurrente en algoritmos de optimización, como el de retropropagación ([Goodfellow et al., 2016](#); [Rumelhart et al., 1986](#)).

2.3 ANÁLISIS FUNCIONAL

Introducimos algunos conceptos del Análisis Funcional, que son cruciales para la demostración del Teorema de Aproximación Universal. Textos de referencia para esta área son [Kreyszig \(1989\)](#) y [Rudin \(1991\)](#).

Definición 18 (Espacio de Funciones Continuas $C(K)$). Sea K un espacio topológico compacto Hausdorff (en nuestro caso, $I_n = [0, 1]^n \subset \mathbb{R}^n$). Definimos el espacio $C(K)$ como el conjunto de todas las funciones continuas $f : K \rightarrow \mathbb{R}$ (o $\mathbb{C}$). Dotamos a $C(K)$ de la estructura de espacio de Banach con la **norma del supremo** (o norma uniforme):

$$\|f\|_{\infty} = \sup_{\mathbf{x} \in K} |f(\mathbf{x})|$$

([Kreyszig, 1989](#)).

Definición 19 (Densidad). Decimos que un subconjunto S de un espacio normado X (e.g., $C(K)$) es **denso** en X si la clausura de S , denotada $\bar{S}$, es igual a X . Esto equivale a decir que para todo $f \in X$ y todo $\epsilon > 0$, existe un $g \in S$ tal que $\|f - g\|_\infty < \epsilon$ (usando la norma del espacio ambiente) (Rudin, 1991).

Definición 20 (Funcional Lineal Acotado). Sea X un espacio vectorial normado sobre $\mathbb{K}$ ($\mathbb{R}$ o $\mathbb{C}$). Definimos un **funcional lineal** como una aplicación $L : X \rightarrow \mathbb{K}$ tal que para todo $x, y \in X$ y todo $\alpha \in \mathbb{K}$:

1. $L(x + y) = L(x) + L(y)$ (aditividad)
2. $L(\alpha x) = \alpha L(x)$ (homogeneidad)

Decimos que un funcional lineal L es **acotado** si existe una constante $M \geq 0$ tal que $|L(x)| \leq M \|x\|$ para todo $x \in X$. El conjunto de todos los funcionales lineales acotados sobre X conforma el espacio dual X^* (Kreyszig, 1989).

Teorema 2 (Teorema de Hahn-Banach (Corolario para subespacios)). (Kreyszig, 1989, Thm. 4.3-2). Sea S un subespacio de un espacio vectorial normado X . Si $f \in X$ y la distancia de f a S , $d(f, S) = \inf_{s \in S} \|f - s\| = \delta > 0$ (i.e., $f \notin \bar{S}$), entonces existe un funcional lineal acotado $L \in X^*$ tal que:

1. $L(s) = 0$ para todo $s \in S$ (i.e., L anula a S).
2. $L(f) = \delta$ (o, más generalmente, $L(f) \neq 0$).
3. $\|L\| = 1$.

En particular, si S es un subespacio propio cerrado de X , existe un $L \in X^*$ no nulo tal que $L(s) = 0$ para todo $s \in S$.

Definición 21 (Medida de Borel Signada Regular). Sea K un espacio topológico compacto Hausdorff. Una **medida de Borel signada** μ sobre K es una función de conjunto $\mu : \mathcal{B}(K) \rightarrow \mathbb{R} \cup \{-\infty, \infty\}$ (donde $\mathcal{B}(K)$ es la σ -álgebra de Borel de K) que es σ -aditiva. Decimos que una medida de Borel signada μ es **regular** si para todo conjunto de Borel B :

$$|\mu(B)| = \inf\{\mu(U) : B \subseteq U, U \text{ abierto}\} \\ = \sup\{\mu(C) : C \subseteq B, C \text{ compacto}\} \quad (\text{para medidas positivas, con ajuste para signadas})$$

El espacio de todas las medidas de Borel signadas regulares finitas sobre K lo denotamos $M(K)$ (Rudin, 1987, Definición 6.2, 6.4).

Teorema 3 (Teorema de Representación de Riesz-Markov-Kakutani). (Rudin, 1987, Thm. 6.19). Sea K un espacio topológico compacto Hausdorff. El espacio dual de $C(K)$, denotado $(C(K))^*$, es isométricamente isomorfo al espacio $M(K)$ de las medidas de Borel signadas regulares finitas sobre K . Es decir, para cada funcional lineal acotado $L \in (C(K))^*$, existe una única medida $\mu \in M(K)$ tal que:

$$L(f) = \int_K f(\mathbf{x}) d\mu(\mathbf{x}) \quad \text{para todo } f \in C(K)$$

y $\|L\| = \|\mu\|_{TV}$ (norma de variación total de μ).

Teorema 4 (Teorema de Convergencia Dominada de Lebesgue). (*Rudin, 1987, Thm. 1.34*). Sea $(X, \mathcal{M}, \mu)$ un espacio de medida. Si $\{f_k\}_{k=1}^{\infty}$ es una secuencia de funciones medibles $f_k : X \rightarrow \mathbb{R}$ tal que $f_k(\mathbf{x}) \rightarrow f(\mathbf{x})$ casi en todas partes para $\mathbf{x} \in X$ cuando $k \rightarrow \infty$, y existe una función integrable $g : X \rightarrow [0, \infty]$ tal que $|f_k(\mathbf{x})| \leq g(\mathbf{x})$ casi en todas partes para todo k , entonces f es integrable y

$$\lim_{k \rightarrow \infty} \int_X f_k d\mu = \int_X f d\mu$$

2.4 PROBABILIDAD Y ESTADÍSTICA

La Probabilidad y la Estadística nos permiten modelar la incertidumbre, describir conjuntos de datos y realizar inferencias. La probabilidad proporciona el lenguaje para describir fenómenos aleatorios, mientras que la estadística se ocupa de la recolección, análisis, interpretación y presentación de datos. Textos de referencia incluyen *Casella and Berger (2002)*, *Ross (2014)*, *DeGroot and Schervish (2012)* y *Rohatgi and Saleh (2015)*.

2.4.1 Estadística Descriptiva

La estadística descriptiva resume y describe las características principales de un conjunto de datos.

Definición 22 (Media Aritmética). Dada una colección finita de N números reales $\{x_1, \dots, x_N\} \subset \mathbb{R}$, definimos la **media aritmética** como $\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$ (*Casella and Berger, 2002*).

Definición 23 (Mediana). Dada una colección finita de N números reales $\{x_1, \dots, x_N\} \subset \mathbb{R}$, y su versión ordenada no decreciente $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(N)}$, definimos la **mediana** Me como el valor central. Formalmente, si N es impar, $Me = x_{((N+1)/2)}$. Si N es par, $Me = \frac{1}{2}(x_{(N/2)} + x_{(N/2+1)})$ (*DeGroot and Schervish, 2012*).

La media y la mediana son medidas de centralización que buscan representar el valor típico de un conjunto de datos. Para medir la dispersión o variabilidad de los datos alrededor de la media, utilizamos la varianza y la desviación estándar.

Definición 24 (Varianza Muestral). Dada una colección finita de N números reales $\{x_1, \dots, x_N\} \subset \mathbb{R}$ con media aritmética $\bar{x}$, definimos la **varianza muestral**, denotada s^2 o $\sigma_{\text{muestra}}^2$, como el promedio de las diferencias al cuadrado con respecto a la media:

$$s^2 = \frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2$$

El uso del denominador $N-1$ en lugar de N proporciona un estimador insesgado de la varianza poblacional cuando los datos se consideran una muestra de una población mayor (Casella and Berger, 2002; Ross, 2014). Si el conjunto de N datos constituye la totalidad de la población, se utiliza N en el denominador para calcular la varianza poblacional σ^2 .

Definición 25 (Desviación Estándar Muestral). Dada una colección finita de N números reales $\{x_1, \dots, x_N\} \subset \mathbb{R}$, la **desviación estándar muestral**, denotada s o σ_{muestra} , es la raíz cuadrada positiva de la varianza muestral:

$$s = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}$$

La desviación estándar se expresa en las mismas unidades que los datos originales, lo que facilita su interpretación como una medida típica de la distancia de las observaciones a la media (DeGroot and Schervish, 2012).

Mientras que la media (Definition 22) y la mediana son medidas de centralización que buscan representar el valor típico de un conjunto de datos, la varianza y la desviación estándar cuantifican la dispersión o variabilidad de esos datos alrededor de la media.

2.4.2 Fundamentos de Probabilidad

La teoría de la probabilidad proporciona un marco para cuantificar la incertidumbre, basada en los axiomas de Kolmogorov (Ross, 2014).

Definición 26 (Espacio de Probabilidad y Eventos). Definimos un **espacio de probabilidad** como una terna $(\Omega, \mathcal{F}, P)$, donde Ω es el **espacio muestral** (el conjunto de todos los posibles resultados de un experimento aleatorio), $\mathcal{F}$ es una σ -álgebra de **eventos** (subconjuntos de Ω a los que podemos asignar una probabilidad), y $P : \mathcal{F} \rightarrow [0, 1]$ es una **medida de probabilidad** que satisface los axiomas de Kolmogorov (Ross, 2014).

Definición 27 (Probabilidad de un Evento). Definimos la **probabilidad** de un evento $A \in \mathcal{F}$, denotada $P(A)$, como un número en el intervalo $[0, 1]$ que cuantifica la verosimilitud de que ocurra A .

Definición 28 (Variable Aleatoria). Definimos una **variable aleatoria** (v.a.) X como una función medible $X : \Omega \rightarrow \mathbb{R}$. Esta transforma los resultados del espacio muestral en valores numéricos. Podemos considerar también vectores aleatorios ($X : \Omega \rightarrow \mathbb{R}^n$) o matrices aleatorias ($X : \Omega \rightarrow \mathbb{R}^{m \times n}$) (Casella and Berger, 2002).

Clasificamos las variables aleatorias como discretas o continuas, dependiendo del conjunto de valores que pueden tomar. A continuación, definiremos cada una de estas variables con el objetivo de caracterizar su comportamiento.

Definición 29 (Variable Aleatoria Discreta). Decimos que una variable aleatoria X es **discreta** si su conjunto de posibles valores es finito o contablemente infinito. Describimos la distribución de probabilidad de una v.a. discreta mediante su **función de masa de probabilidad** (FMP), $p_X(x) = P(X = x)$, que asigna una probabilidad a cada valor posible x . La FMP satisface $p_X(x) \geq 0$ para todo x y $\sum_x p_X(x) = 1$ (Ross, 2014).

Definición 30 (Variable Aleatoria Continua). Decimos que una variable aleatoria X es **continua** si toma valores en un intervalo de números reales (o $\mathbb{R}$). Describimos la distribución de probabilidad de una v.a. continua mediante su **función de densidad de probabilidad** (FDP), $f_X(x)$, tal que $P(a \leq X \leq b) = \int_a^b f_X(x)dx$ para cualesquiera $a \leq b$. La FDP satisface $f_X(x) \geq 0$ para todo x y $\int_{-\infty}^{\infty} f_X(x)dx = 1$ (DeGroot and Schervish, 2012).

Un concepto crucial es cómo la información sobre un evento afecta la probabilidad de otro.

Definición 31 (Probabilidad Condicionada). Dados dos eventos $A, B \in \mathcal{F}$ con $P(B) > 0$, definimos la **probabilidad condicionada** de A dado B , denotada $P(A|B)$, como:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Esta representa la probabilidad de que ocurra A sabiendo que B ha ocurrido (Ross, 2014).

Relacionamos la probabilidad condicionada con el concepto de independencia.

Definición 32 (Independencia Estadística). Decimos que dos eventos $A, B \in \mathcal{F}$ son **estadísticamente independientes** si la ocurrencia de uno no altera la probabilidad del otro. Formalizamos esto como $P(A \cap B) = P(A)P(B)$. Si $P(B) > 0$, una definición equivalente es $P(A|B) = P(A)$. Decimos que dos variables aleatorias X e Y son independientes si para cualesquiera conjuntos medibles $A, B \subseteq \mathbb{R}$, los eventos $\{X \in A\}$ e $\{Y \in B\}$ son independientes (Casella and Berger, 2002).

Demostración de la equivalencia de las definiciones de independencia (si $P(B) > 0$). Queremos demostrar que $P(A \cap B) = P(A)P(B) \iff P(A|B) = P(A)$, asumiendo $P(B) > 0$.

($\Rightarrow$): Supongamos que $P(A \cap B) = P(A)P(B)$. Por la definición de probabilidad condicionada (Definition 31), tenemos:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Sustituyendo la hipótesis $P(A \cap B) = P(A)P(B)$:

$$P(A|B) = \frac{P(A)P(B)}{P(B)}$$

Como $P(B) > 0$, podemos cancelar $P(B)$ en el numerador y denominador:

$$P(A|B) = P(A)$$

($\Leftarrow$): Supongamos que $P(A|B) = P(A)$. Por la definición de probabilidad condicionada (Definition 31):

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Sustituyendo la hipótesis $P(A|B) = P(A)$:

$$P(A) = \frac{P(A \cap B)}{P(B)}$$

Multiplicando ambos lados por $P(B)$ (que es positivo y, por lo tanto, no cero):

$$P(A)P(B) = P(A \cap B)$$

Esto completa la demostración de la equivalencia, como se puede encontrar en textos estándar de probabilidad (Ross, 2014). $\square$

2.4.3 Distribuciones de Probabilidad Discretas Comunes

Presentamos algunas distribuciones discretas fundamentales (Casella and Berger, 2002; Ross, 2014).

Definición 33 (Distribución Binomial). La distribución Binomial describe el número de éxitos k en una secuencia de n ensayos de Bernoulli independientes e idénticamente distribuidos, cada uno con una probabilidad de éxito p .

Sea X una variable aleatoria que sigue una distribución Binomial, que denotamos como $X \sim \text{Binomial}(n, p)$ o $X \sim B(n, p)$. La **función de masa de probabilidad** (FMP) de X para obtener exactamente k éxitos en n ensayos es:

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

para $k = 0, 1, 2, \dots, n$, donde:

- n es el número total de ensayos (un entero no negativo).
- k es el número de éxitos (un entero no negativo, $0 \leq k \leq n$).
- p es la probabilidad de éxito en un solo ensayo ($0 \leq p \leq 1$).
- $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ es el coeficiente binomial.

La probabilidad de fracaso en un solo ensayo es $q = 1 - p$ (Ross, 2014).

Definición 34 (Distribución Multinomial). La distribución Multinomial generaliza la distribución Binomial. Describe la probabilidad de obtener un conjunto específico de conteos para K categorías mutuamente excluyentes, después de realizar n ensayos independientes.

Sean $X_1, X_2, \dots, X_K$ variables aleatorias que representan el número de veces que ocurre cada una de las K categorías en n ensayos. El vector de conteos $\mathbf{x} = (x_1, x_2, \dots, x_K)^T$ sigue una distribución Multinomial con parámetros n (número de ensayos) y $\mathbf{p} = (p_1, p_2, \dots, p_K)^T$ (vector de probabilidades de cada categoría), que denotamos como $\mathbf{X} \sim \text{Multinomial}(n, \mathbf{p})$. Las condiciones son:

- x_i es el número de ocurrencias de la categoría i (entero no negativo).
- $\sum_{i=1}^K x_i = n$.
- p_i es la probabilidad de que un ensayo resulte en la categoría i .
- $\sum_{i=1}^K p_i = 1$ y $p_i \geq 0$ para todo $i = 1, \dots, K$.

La **función de masa de probabilidad** (FMP) conjunta de $X_1, \dots, X_K$ es:

$$P(X_1 = x_1, X_2 = x_2, \dots, X_K = x_K) = \frac{n!}{x_1! x_2! \dots x_K!} p_1^{x_1} p_2^{x_2} \dots p_K^{x_K}$$

donde $\frac{n!}{x_1! x_2! \dots x_K!}$ es el coeficiente multinomial. Si $K = 2$, la distribución Multinomial se reduce a la Definición 33 (con $x_1 = k$, $x_2 = n - k$, $p_1 = p$, $p_2 = 1 - p$) (Casella and Berger, 2002).

Definición 35 (Distribución Categórica). La distribución Categórica (también conocida como distribución de Bernoulli generalizada o Multinoulli) es un caso especial de la Definición 34 donde realizamos un solo ensayo, es decir, $n = 1$. Describe la probabilidad de que un solo evento pertenezca a una de K categorías mutuamente excluyentes (Bishop, 2006).

Sea Y una variable aleatoria que sigue una distribución Categórica con parámetro $\mathbf{p} = (p_1, p_2, \dots, p_K)^T$, que denotamos como $Y \sim \text{Categorical}(\mathbf{p})$. Las condiciones son:

- K es el número de categorías posibles.
- p_i es la probabilidad de que el resultado sea la categoría i .
- $\sum_{i=1}^K p_i = 1$ y $p_i \geq 0$ para todo $i = 1, \dots, K$.

Estos fundamentos probabilísticos son indispensables para que comprendamos la aleatoriedad inherente en los datos y en los procesos de modelado.

2.5 FUNCIONES ESPECÍFICAS: LA FUNCIÓN SOFTMAX

Presentamos funciones no lineales con propiedades particulares que son relevantes en diversas aplicaciones, como la función Softmax, ampliamente utilizada en clasificación multiclase (Bishop, 2006). La **Softmax** (también llamada función Softmax) transforma un vector de números reales en un vector de números positivos que suman 1.

Definición 36 (Softmax). Definimos la **Softmax** $\sigma : \mathbb{R}^K \rightarrow \mathbb{R}^K$ como la función que toma un vector de entrada $\mathbf{z} = (z_1, \dots, z_K)^T \in \mathbb{R}^K$ y devuelve un vector de salida $\sigma(\mathbf{z}) = (p_1, \dots, p_K)^T$ con componentes:

$$p_i = [\sigma(\mathbf{z})]_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

para $i = 1, \dots, K$

La Softmax posee dos propiedades clave que la relacionan con las distribuciones de probabilidad discretas.

Proposición 5 (Propiedades de la Softmax). Para el vector de salida $\mathbf{p} = \sigma(\mathbf{z})$, se cumplen (Bishop, 2006):

1. Positividad: $p_i > 0$ para todo $i = 1, \dots, K$.
2. Normalización: $\sum_{i=1}^K p_i = 1$.

Demostración de la Propiedad 5.1 (Positividad). Para cualquier número real z_i , la función exponencial e^{z_i} es estrictamente positiva. La suma en el denominador, $\sum_{j=1}^K e^{z_j}$, es una suma de términos estrictamente positivos, por lo que también es estrictamente positiva. El cociente de dos números estrictamente positivos es estrictamente positivo. Por lo tanto, concluimos que $p_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} > 0$ para todo i . $\square$

Demostración de la Propiedad 5.2 (Normalización). Calculamos la suma de las componentes del vector de salida:

$$\sum_{i=1}^K p_i = \sum_{i=1}^K \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Dado que el denominador $\sum_{j=1}^K e^{z_j}$ es una constante con respecto al índice de suma i , podemos sacarlo fuera de la suma:

$$\sum_{i=1}^K p_i = \frac{1}{\sum_{j=1}^K e^{z_j}} \left(\sum_{i=1}^K e^{z_i} \right)$$

Observamos que la suma en el numerador $\sum_{i=1}^K e^{z_i}$ es idéntica a la suma en el denominador $\sum_{j=1}^K e^{z_j}$ (solo cambia el nombre del índice de sumación, lo cual no altera

el resultado de la suma sobre todos los componentes). Por lo tanto, simplificamos la expresión a:

$$\frac{\sum_{i=1}^K e^{z_i}}{\sum_{j=1}^K e^{z_j}} = 1$$

Esto completa la demostración. $\square$

2.5.1 Apunte sobre estabilidad numérica

Para mejorar la estabilidad numérica al calcular la Softmax, especialmente cuando los valores de entrada x_i son grandes (lo que podría llevar a que e^{x_i} cause un desbordamiento), restamos el valor máximo del vector de entrada de cada elemento *antes* de aplicar la exponencial. Esta es una práctica común (ver, Capítulo 4)

Definimos $m = \max\{x_1, x_2, \dots, x_N\}$. Entonces, calculamos:

$$\text{softmax}(x_i) = \frac{e^{x_i - m}}{\sum_{j=1}^N e^{x_j - m}} = \frac{e^{-m} \sum_{i=1}^N e^{x_i}}{e^{-m} \sum_{j=1}^N e^{x_j}} = \frac{\sum_{i=1}^N e^{x_i}}{\sum_{j=1}^N e^{x_j}}$$

Esto es matemáticamente equivalente pero numéricamente estable. Al restar m , el exponente más grande será 0 (ya que para algún k , $x_k = m$, entonces $x_k - m = 0$), y $e^0 = 1$. Todos los demás exponentes serán negativos, resultando en valores $e^{x_j - m}$ entre $(0, 1]$. Esto evita el desbordamiento de los términos exponenciales y también ayuda a prevenir el subdesbordamiento en el denominador (ya que al menos un término en la suma será 1).

Ejemplo numérico breve con $K=3$

Consideremos un vector de logits (valores de entrada) $\mathbf{x} = (1, 2, 5)^T$.

■ **Cálculo estándar (potencialmente inestable):**

- $e^1 \approx 2,718$
- $e^2 \approx 7,389$
- $e^5 \approx 148,413$

Suma de exponenciales: $S_E = 2,718 + 7,389 + 148,413 = 158,520$.

$$\text{softmax}(x_1) = \frac{2,718}{158,520} \approx 0,017$$

$$\text{softmax}(x_2) = \frac{7,389}{158,520} \approx 0,047$$

$$\text{softmax}(x_3) = \frac{148,413}{158,520} \approx 0,936$$

Vector de salida: $(0,017, 0,047, 0,936)^T$. (Suma $\approx 1,000$)

- **Cálculo numéricamente estable:** El valor máximo es $m = \max(1, 2, 5) = 5$. Restamos m de cada elemento: $\mathbf{x}' = (1 - 5, 2 - 5, 5 - 5)^T = (-4, -3, 0)^T$. Calculamos las exponenciales de $\mathbf{x}'$:

- $e^{-4} \approx 0,0183$
- $e^{-3} \approx 0,0498$
- $e^0 = 1,0000$

Suma de nuevas exponenciales: $S'_E = 0,0183 + 0,0498 + 1,0000 = 1,0681$.

$$\text{softmax}(x_1) = \frac{0,0183}{1,0681} \approx 0,017$$

$$\text{softmax}(x_2) = \frac{0,0498}{1,0681} \approx 0,047$$

$$\text{softmax}(x_3) = \frac{1,0000}{1,0681} \approx 0,936$$

Vector de salida: $(0,017, 0,047, 0,936)^T$. (Suma $\approx 1,000$)

Ambos métodos producen el mismo resultado (dentro de la precisión de redondeo). Sin embargo, si los valores de entrada fueran, por ejemplo, $(100, 102, 105)^T$, e^{105} excedería el límite de flotantes estándar, mientras que el método estable operaría con e^0, e^{-3}, e^{-5} , que son numéricamente manejables.

El vector de salida de la función Softmax, que denotamos como $\mathbf{y} = (y_1, \dots, y_K)^T$ donde cada componente $y_i = \text{softmax}(x_i) = \frac{e^{x_i - m}}{\sum_{j=1}^K e^{x_j - m}}$, tiene elementos y_i que son no negativos ($y_i \geq 0$) y suman a uno ($\sum_{i=1}^K y_i = 1$). Estas propiedades fundamentales (no negatividad y suma igual a uno) permiten que interpretemos el vector $\mathbf{y}$ como los parámetros de una distribución de probabilidad categórica (Definition 35) sobre K posibles resultados mutuamente excluyentes (Bishop, 2006).

2.6 MÉTODOS NUMÉRICOS

Los métodos numéricos desarrollan algoritmos para resolver problemas matemáticos que no podemos resolver analíticamente de manera sencilla o eficiente. Estos métodos son esenciales en la práctica computacional, como detallan Burden and Faires (2011) y Kincaid and Cheney (2002). Una técnica común es la interpolación, que estima valores desconocidos a partir de valores conocidos.

2.6.1 Interpolación lineal

Definimos la interpolación lineal como la forma más simple de interpolación, que aproxima una función entre dos puntos conocidos mediante una línea recta (Burden and Faires, 2011).

Definición 37 (Interpolación Lineal). Dados dos puntos (x_0, y_0) y (x_1, y_1) en el plano cartesiano $\mathbb{R}^2$, con $x_0 \neq x_1$, la función de **interpolación lineal** que pasa por estos puntos es la única línea recta. Para cualquier valor x en el intervalo $[x_0, x_1]$, calculamos el valor interpolado $f(x)$ como:

$$f(x) = y_0 + \frac{x - x_0}{x_1 - x_0}(y_1 - y_0)$$

Podemos reescribir esta fórmula como una combinación convexa de y_0 e y_1 :

$$f(x) = (1 - \alpha)y_0 + \alpha y_1$$

donde $\alpha = \frac{x - x_0}{x_1 - x_0}$. Notamos que α varía de 0 a 1 a medida que x varía de x_0 a x_1 .

Derivación de la fórmula de interpolación lineal. Buscamos la ecuación de la recta $y = mx + c$ que pasa por los puntos (x_0, y_0) y (x_1, y_1) . Calculamos la pendiente m de la recta como:

$$m = \frac{y_1 - y_0}{x_1 - x_0}$$

Usamos la forma punto-pendiente de la ecuación de una recta con el punto (x_0, y_0) :

$$y - y_0 = m(x - x_0)$$

Sustituyendo el valor de m :

$$y - y_0 = \frac{y_1 - y_0}{x_1 - x_0}(x - x_0)$$

Despejando y , que representa el valor interpolado $f(x)$:

$$f(x) = y = y_0 + \frac{x - x_0}{x_1 - x_0}(y_1 - y_0)$$

Esta es la primera fórmula dada en la Definition 37.

Para obtener la segunda forma, definimos $\alpha = \frac{x - x_0}{x_1 - x_0}$. Entonces:

$$f(x) = y_0 + \alpha(y_1 - y_0)$$

$$f(x) = y_0 + \alpha y_1 - \alpha y_0$$

$$f(x) = (1 - \alpha)y_0 + \alpha y_1$$

Esta es la segunda fórmula. Cuando $x = x_0$, $\alpha = \frac{x_0 - x_0}{x_1 - x_0} = 0$, y $f(x_0) = (1 - 0)y_0 + 0 \cdot y_1 = y_0$. Cuando $x = x_1$, $\alpha = \frac{x_1 - x_0}{x_1 - x_0} = 1$, y $f(x_1) = (1 - 1)y_0 + 1 \cdot y_1 = y_1$. La fórmula interpola correctamente los puntos dados. $\square$

Utilizamos este método comúnmente para estimar valores intermedios cuando solo conocemos puntos discretos.

2.6.2 Limitaciones de la Interpolación Lineal

Si bien la interpolación lineal es simple y computacionalmente económica, presenta varias limitaciones (Burden and Faires, 2011; Kincaid and Cheney, 2002):

- **No es suave (diferenciable en los nodos):** La función interpolada resulta continua, pero su derivada es discontinua en los puntos de datos originales $(x_0, x_1, \dots, x_n)$. Esto significa que la función resultante no es suave y puede presentar esquinas o quiebres en los nodos. Esto puede ser problemático si requerimos derivadas de orden superior.
- **Presupone linealidad:** El método asume que la función subyacente se comporta de manera lineal entre los puntos de datos conocidos. Si la función real tiene una curvatura significativa o variaciones no lineales entre los puntos, la interpolación lineal producirá una aproximación pobre.
- **Pobre para la extrapolación:** Diseñamos la interpolación lineal para estimar valores *dentro* del rango de los datos conocidos ($[x_0, x_1]$ en el caso de dos puntos, o $[\min(x_i), \max(x_i)]$ para múltiples puntos). Usarla para extrapolar valores *fuera* de este rango puede llevar a errores muy grandes, ya que no tenemos información sobre el comportamiento de la función más allá de los puntos observados.
- **Fenómeno de Runge (con interpolación polinómica de alto grado, no lineal directa):** Aunque la interpolación lineal simple entre dos puntos no sufre directamente del fenómeno de Runge, si intentamos construir un único polinomio de alto grado que pase por *muchos* puntos de datos (en lugar de interpolar linealmente por partes), pueden aparecer oscilaciones espurias entre los puntos, especialmente cerca de los extremos del intervalo. La interpolación lineal por partes evita este problema específico (Burden and Faires, 2011).
- **Error de aproximación:** El error de la interpolación lineal depende de la distancia entre los puntos de datos y de la segunda derivada de la función subyacente. Si los puntos están muy espaciados o la función tiene una curvatura alta, el error puede ser considerable.

A pesar de estas limitaciones, su simplicidad la hace una herramienta útil para aproximaciones rápidas o cuando sabemos que la función subyacente es aproximadamente lineal entre los puntos muestreados. Para funciones más complejas o cuando requerimos mayor suavidad, solemos preferir métodos de interpolación de orden superior como la interpolación polinómica (e.g., Lagrange, Newton), splines, o interpolación trigonométrica (Kincaid and Cheney, 2002).

2.7 TEOREMA DE APROXIMACIÓN UNIVERSAL (CYBENKO)

Establecemos el Teorema de Aproximación Universal de [Cybenko \(1989\)](#), que es un resultado fundamental en la teoría de redes neuronales. Sea $I_n = [0, 1]^n$ el hipercubo unitario n-dimensional. Nuestro objetivo es demostrar que las funciones de la forma:

$$G(\mathbf{x}) = \sum_{j=1}^N \alpha_j \sigma(\mathbf{y}_j^T \mathbf{x} + \theta_j)$$

donde $\mathbf{x}, \mathbf{y}_j \in \mathbb{R}^n$, $\alpha_j, \theta_j \in \mathbb{R}$, $N \in \mathbb{N}$, y σ es una función de activación adecuada, son densas en $C(I_n)$.

Definición 38 (Función Sigmoidal (según Cybenko)). Definimos una función $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ como **sigmoidal** ([Cybenko, 1989](#)) si

$$\sigma(t) \rightarrow \begin{cases} 1 & \text{si } t \rightarrow +\infty \\ 0 & \text{si } t \rightarrow -\infty \end{cases}$$

[Cybenko \(1989\)](#) considera funciones sigmoides continuas.

Definición 39 (Función Discriminatoria (según Cybenko)). Definimos una función continua $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ como **discriminatoria** ([Cybenko, 1989](#)) si para cualquier medida $\mu \in M(I_n)$ (medida de Borel signada regular finita sobre I_n), la condición

$$\int_{I_n} \sigma(\mathbf{y}^T \mathbf{x} + \theta) d\mu(\mathbf{x}) = 0 \quad \text{para todo } \mathbf{y} \in \mathbb{R}^n, \theta \in \mathbb{R}$$

implica que $\mu = 0$ (la medida nula).

Lema 1 (Lema 1 de Cybenko: Sigmoidal $\implies$ Discriminatoria). ([Cybenko, 1989, Lemma 1](#)). Toda función $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ continua y sigmoidal (según Def. 38) es discriminatoria (según Def. 39).

Esquema de la Demostración del Lema 1 (basado en [Cybenko \(1989\)](#)). 1. Asumimos que

1. $\int_{I_n} \sigma(\mathbf{y}^T \mathbf{x} + \theta) d\mu(\mathbf{x}) = 0$ para todo $\mathbf{y}, \theta$.
2. Consideramos la familia de funciones $\sigma_{\lambda, \phi}(u) = \sigma(\lambda u + \phi)$ para $\lambda > 0$. Entonces $\int_{I_n} \sigma(\lambda(\mathbf{y}^T \mathbf{x} + \theta_0) + \phi) d\mu(\mathbf{x}) = 0$ para $\mathbf{y}, \theta_0, \phi$ fijos y todo λ . (Obtenemos esto de la hipótesis tomando un $\mathbf{y}'$ adecuado y θ').
3. Sea $u = \mathbf{y}^T \mathbf{x} + \theta_0$. A medida que $\lambda \rightarrow \infty$:

$$\sigma(\lambda u + \phi) \rightarrow \gamma_\phi(u) = \begin{cases} 1 & \text{si } u > 0 \\ 0 & \text{si } u < 0 \\ \sigma(\phi) & \text{si } u = 0 \end{cases}$$

Esta convergencia es puntual. Como σ es continua y sigmoideal, es acotada. Por el Teorema de Convergencia Dominada de Lebesgue (Theorem 4, Rudin 1987), podemos intercambiar el límite y la integral:

$$\int_{I_n} \gamma_\phi(\mathbf{y}^T \mathbf{x} + \theta_0) d\mu(\mathbf{x}) = 0$$

Esto es $\mu(H_{\mathbf{y},\theta_0}^+) + \sigma(\phi)\mu(\Pi_{\mathbf{y},\theta_0}) = 0$, donde $H_{\mathbf{y},\theta_0}^+ = \{\mathbf{x} \in I_n : \mathbf{y}^T \mathbf{x} + \theta_0 > 0\}$ y $\Pi_{\mathbf{y},\theta_0} = \{\mathbf{x} \in I_n : \mathbf{y}^T \mathbf{x} + \theta_0 = 0\}$.

4. Tomando el límite cuando $\phi \rightarrow -\infty$, tenemos $\sigma(\phi) \rightarrow 0$. Entonces $\mu(H_{\mathbf{y},\theta_0}^+) = 0$.
5. De forma similar, tomando $\phi \rightarrow +\infty$, $\sigma(\phi) \rightarrow 1$. Usando una transformación adecuada ($1 - \sigma$ en lugar de σ para definir una función sigmoideal que va de 1 a 0, y ajustando los argumentos), podemos mostrar que $\mu(H_{\mathbf{y},\theta_0}^-) = 0$, donde $H_{\mathbf{y},\theta_0}^- = \{\mathbf{x} \in I_n : \mathbf{y}^T \mathbf{x} + \theta_0 < 0\}$.
6. Dado que $\mu(I_n) = \mu(H_{\mathbf{y},\theta_0}^+) + \mu(H_{\mathbf{y},\theta_0}^-) + \mu(\Pi_{\mathbf{y},\theta_0})$, deducimos que $\mu(\Pi_{\mathbf{y},\theta_0})$ también debe ser cero para casi todo θ_0 si $\mathbf{y} \neq \mathbf{0}$.
7. El hecho de que la medida de todos los semiplanos abiertos sea cero ($\mu(H_{\mathbf{y},\theta_0}^+) = 0$ y $\mu(H_{\mathbf{y},\theta_0}^-) = 0$ para todo $\mathbf{y} \neq \mathbf{0}$ y casi todo θ_0) implica que la Transformada de Fourier de la medida μ es idénticamente cero para $\boldsymbol{\xi} \neq \mathbf{0}$.

$$\hat{\mu}(\boldsymbol{\xi}) = \int_{I_n} e^{-2\pi i \boldsymbol{\xi}^T \mathbf{x}} d\mu(\mathbf{x}) = 0 \quad \text{para todo } \boldsymbol{\xi} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$$

(Este paso requiere un argumento técnico que Cybenko (1989) detalla, mostrando que la integral de $e^{i\boldsymbol{\xi}^T \mathbf{x}}$ contra $d\mu(\mathbf{x})$ es cero usando la relación con las funciones indicadoras de semiplanos. La anulación para $\boldsymbol{\xi} = \mathbf{0}$ se sigue de $\mu(I_n) = 0$, que se deriva de los pasos anteriores).

8. Si la Transformada de Fourier de una medida finita es cero, entonces la medida misma es cero ($\mu = 0$) por la unicidad de la Transformada de Fourier (Rudin, 1987, Capítulo 9).
9. Por lo tanto, concluimos que σ es discriminatoria.

□

Teorema 5 (Teorema 1 de Cybenko - Teorema de Aproximación Universal). (Cybenko, 1989, Theorem 1). Sea $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ una función continua y discriminatoria (por Lema 1, esto incluye a las sigmoides continuas). Entonces, las sumas finitas de la forma

$$G(\mathbf{x}) = \sum_{j=1}^N \alpha_j \sigma(\mathbf{y}_j^T \mathbf{x} + \theta_j)$$

son densas en $C(I_n)$ con respecto a la norma del supremo.

Esquema de la Demostración del Teorema 5 (basado en Cybenko (1989)). 1. Sea S el conjunto de todas las funciones $G(\mathbf{x})$ de la forma dada. S es un subespacio vectorial de $C(I_n)$ (no necesariamente cerrado).

2. Queremos demostrar que $\bar{S} = C(I_n)$. Procedemos por contradicción.
3. Suponemos que $\bar{S} \neq C(I_n)$. Entonces $\bar{S}$ es un subespacio propio cerrado de $C(I_n)$.
4. Por el Teorema de Hahn-Banach (Theorem 2, Kreyszig 1989), existe un funcional lineal acotado $L \in (C(I_n))^*$ tal que $L \neq 0$ (el funcional nulo) y $L(g) = 0$ para todo $g \in \bar{S}$ (y por lo tanto para todo $g \in S$).
5. Por el Teorema de Representación de Riesz-Markov-Kakutani (Theorem 3, Rudin 1987), existe una medida de Borel signada regular finita no nula $\mu \in M(I_n)$ (ya que $L \neq 0$) tal que para toda función $h \in C(I_n)$:

$$L(h) = \int_{I_n} h(\mathbf{x}) d\mu(\mathbf{x})$$

6. En particular, para cualquier función de la forma $s(\mathbf{x}) = \sigma(\mathbf{y}^T \mathbf{x} + \theta)$ (que es un elemento de S tomando $N = 1, \alpha_1 = 1$), tenemos que $L(s(\mathbf{x})) = 0$. Es decir:

$$\int_{I_n} \sigma(\mathbf{y}^T \mathbf{x} + \theta) d\mu(\mathbf{x}) = 0 \quad \text{para todo } \mathbf{y} \in \mathbb{R}^n, \theta \in \mathbb{R}$$

7. Dado que asumimos que σ es una función discriminatoria (Definition 39), la condición anterior implica que la medida μ debe ser la medida nula ($\mu = 0$).
8. Si $\mu = 0$, entonces el funcional L debe ser el funcional nulo ($L(h) = \int h d0 = 0$ para todo h). Esto contradice el paso 4, donde establecimos que $L \neq 0$.
9. Por lo tanto, la suposición inicial (que $\bar{S} \neq C(I_n)$) debe ser falsa.
10. Concluimos que $\bar{S} = C(I_n)$, es decir, las sumas finitas de la forma $G(\mathbf{x})$ son densas en $C(I_n)$.

□

DEL APRENDIZAJE AUTOMÁTICO AL APRENDIZAJE PROFUNDO: FUNDAMENTOS

En este capítulo sentaremos las bases conceptuales necesarias para comprender el Aprendizaje Profundo (Deep Learning), comenzando por los fundamentos del campo más amplio al que pertenece: el Aprendizaje Automático (Machine Learning). Exploraremos qué significa aprender para una máquina, los diferentes modos en que puede hacerlo y algunos de los modelos y algoritmos más elementales que sirvieron de inspiración para desarrollos posteriores.

3.1 ¿QUÉ ES EL APRENDIZAJE AUTOMÁTICO?

El Aprendizaje Automático (Machine Learning, ML) es una rama de la inteligencia artificial y de las ciencias de la computación que se centra en el uso de datos y algoritmos para imitar la forma en que los humanos aprenden, mejorando gradualmente su precisión. Dicho de otro modo, dota a los sistemas de la capacidad de aprender y mejorar a partir de la experiencia (datos) sin ser explícitamente programados para cada tarea específica.

La esencia del aprendizaje automático reside en descubrir patrones subyacentes en los datos. Dada una tarea y un conjunto de datos relevantes, el objetivo es desarrollar un modelo matemático que pueda generalizar su aprendizaje a nuevos datos no vistos previamente, permitiendo realizar predicciones o tomar decisiones informadas. Los componentes fundamentales de un problema de aprendizaje son, como se detalla en [Abu-Mostafa et al. \(2012\)](#):

- El **conjunto de datos (Dataset)**: Colección de ejemplos (observaciones, instancias) que sirven como base empírica para el aprendizaje. Cada ejemplo suele consistir en un conjunto de características (features) y, en algunos paradigmas, una etiqueta o valor objetivo asociado.
- El **modelo o hipótesis (Model/Hypothesis)**: Una representación matemática (por ejemplo, una función o una distribución de probabilidad) que intenta capturar la relación entre las entradas y las salidas, o la estructura inherente de

los datos. El modelo pertenece a un conjunto predefinido de posibles modelos, denominado *espacio de hipótesis* ($\mathcal{H}$).

- **El algoritmo de aprendizaje (Learning Algorithm):** El procedimiento utilizado para ajustar los parámetros del modelo (o seleccionar la mejor hipótesis dentro del espacio $\mathcal{H}$) basándose en los patrones identificados en los datos de entrenamiento.

Este capítulo se centrará en los fundamentos de estos componentes, estableciendo el marco necesario para comprender técnicas más avanzadas.

3.2 PARADIGMAS DEL APRENDIZAJE AUTOMÁTICO

Existen diferentes enfoques o paradigmas dentro del aprendizaje automático, clasificados principalmente según la naturaleza de los datos de entrenamiento y el tipo de retroalimentación disponible durante el aprendizaje. Los dos paradigmas más extendidos son el aprendizaje supervisado y el no supervisado.

3.2.1 Aprendizaje Supervisado

En el aprendizaje supervisado, el algoritmo aprende a partir de un conjunto de datos de entrenamiento donde cada ejemplo $(\mathbf{x}_n, y_n)$ consiste en un vector de características de entrada $\mathbf{x}_n$ y una *etiqueta* o *valor objetivo* deseado y_n . Es decir, se proporciona la respuesta correcta para cada entrada del entrenamiento. El objetivo es aprender una función de mapeo $g : \mathcal{X} \rightarrow \mathcal{Y}$ que se aproxime a la función objetivo desconocida f y que pueda predecir la salida para nuevas entradas $\mathbf{x}$ no vistas previamente.

Dentro del aprendizaje supervisado, distinguimos principalmente dos tipos de tareas:

- **Clasificación:** La salida deseada y es una etiqueta categórica discreta perteneciente a un conjunto finito de clases. El objetivo es asignar nuevas entradas a una de estas categorías predefinidas. Ejemplos incluyen la detección de spam (spam/no spam) o el reconocimiento de dígitos escritos a mano (clases 0 a 9).
- **Regresión:** La salida deseada y es un valor continuo o real. El objetivo es predecir un valor numérico para nuevas entradas. Ejemplos son la predicción del precio de una acción o la estimación de la temperatura.

Los modelos como la Regresión Logística o las Máquinas de Soporte Vectorial (SVM) son ejemplos típicos aplicados a clasificación, mientras que la Regresión Lineal lo es para tareas de regresión.

3.2.2 Aprendizaje No Supervisado

En el aprendizaje no supervisado, el algoritmo trabaja con datos de entrada $\{\mathbf{x}_n\}$ que *no tienen etiquetas* u objetivos predefinidos. El objetivo es descubrir patrones, estructuras o relaciones inherentes en los propios datos sin una guía explícita sobre cuál debería ser la "salida correcta".

Las tareas más comunes dentro del aprendizaje no supervisado incluyen:

- **Clustering (Agrupamiento):** Agrupar los datos en clústeres basados en su similitud intrínseca, de forma que los puntos dentro de un mismo clúster sean más similares entre sí que con los puntos de otros clústeres.
- **Reducción de Dimensionalidad:** Encontrar una representación de los datos en un espacio de menor dimensión, preservando la mayor cantidad de información relevante posible. El Análisis de Componentes Principales (PCA) es un ejemplo clásico.
- **Estimación de Densidad:** Construir un modelo de la distribución de probabilidad subyacente de los datos.

El aprendizaje no supervisado es fundamental para explorar datos y puede servir como paso previo para tareas supervisadas.

3.3 EVALUACIÓN DE MODELOS Y TRATAMIENTO DE DATOS FALTANTES

Una vez que un algoritmo de aprendizaje ha generado una hipótesis final g , es crucial evaluar su rendimiento para entender su capacidad de generalización a datos nuevos. Además, en la práctica, es común encontrar datos incompletos, lo que requiere técnicas específicas para su manejo.

3.3.1 Métricas de Evaluación

La elección de la métrica de evaluación depende intrínsecamente de la tarea (clasificación o regresión) y de los objetivos específicos del problema. Un buen modelo no solo debe ajustarse bien a los datos de entrenamiento (minimizar el error *in-sample*, E_{in}), sino, y más importante, generalizar bien a datos no vistos (minimizar el error *out-of-sample*, E_{out}). Algunas métricas comunes son:

Para Clasificación:

- **Accuracy (Exactitud):** Proporción de predicciones correctas. Simple pero potencialmente engañosa en datasets desbalanceados.
- **Matriz de Confusión:** Tabla que desglosa los aciertos y errores por clase (Verdaderos Positivos/Negativos, Falsos Positivos/Negativos).

- *Precisión, Recall (Sensibilidad) y F1-Score*: Métricas derivadas de la matriz de confusión, más informativas en casos desbalanceados. La precisión mide la fracción de predicciones positivas que fueron correctas. El recall mide la fracción de positivos reales que se identificaron correctamente. El F1-Score es la media armónica de ambas.
- *Área Bajo la Curva ROC (AUC-ROC)*: Evalúa la capacidad del modelo para discriminar entre clases a diferentes umbrales de decisión.

Para Regresión:

- *Error Cuadrático Medio (MSE)*: Promedio de los errores al cuadrado. Sensible a outliers.
- *Raíz del Error Cuadrático Medio (RMSE)*: Raíz cuadrada del MSE, expresada en las mismas unidades que la variable objetivo.
- *Error Absoluto Medio (MAE)*: Promedio de los errores absolutos. Más robusto a outliers que el MSE.
- *Coefficiente de Determinación (R^2)*: Proporción de la varianza de la variable objetivo explicada por el modelo.

La selección y correcta interpretación de estas métricas es fundamental para el desarrollo y despliegue de modelos de ML.

3.4 MODELOS LINEALES FUNDAMENTALES

Los modelos lineales son un pilar fundamental del aprendizaje automático. Su simplicidad matemática permite un análisis profundo y, a menudo, soluciones analíticas, sirviendo además como base para modelos más complejos como las redes neuronales profundas.

3.4.1 El Perceptrón

El Perceptrón es un modelo histórico para clasificación binaria. Representa una neurona artificial que calcula una suma ponderada de sus entradas y aplica una función de activación escalón (la función signo) para producir una salida binaria (+1 o -1). Para una entrada $\mathbf{x}$ (aumentada con $x_0 = 1$), la hipótesis es:

$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x})$$

El *Algoritmo de Aprendizaje del Perceptrón (PLA)* ajusta iterativamente el vector de pesos $\mathbf{w}$ para corregir ejemplos mal clasificados, convergiendo a una solución si los datos son linealmente separables (Abu-Mostafa et al., 2012). Aunque limitado, el Perceptrón introdujo conceptos clave como el aprendizaje iterativo basado en errores (Abu-Mostafa et al., 2012).

3.4.2 Regresión Lineal

La regresión lineal busca modelar una variable objetivo real y como una combinación lineal de las características de entrada $\mathbf{x}$:

$$h(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$$

El objetivo es encontrar el vector de pesos $\mathbf{w}$ que minimice el error cuadrático medio (MSE) en el conjunto de entrenamiento:

$$E_{in}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N (\mathbf{w}^T \mathbf{x}_n - y_n)^2$$

Como vimos, esta minimización tiene una solución analítica en forma cerrada dada por las *ecuaciones normales* (Bishop, 2006):

$$\mathbf{w}_{lin} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

donde $\mathbf{X}$ es la matriz de diseño (datos de entrada por filas) y $\mathbf{y}$ el vector de valores objetivo. La regresión lineal es fundamental en estadística y ML, y variantes como la regresión regularizada (Ridge, Lasso) abordan problemas como la multicolinealidad o la selección de características.

3.4.3 Descenso del Gradiente

Para muchos modelos, incluyendo la regresión logística y las redes neuronales, no existe una solución analítica cerrada para minimizar la función de error. El *descenso del gradiente* es un algoritmo iterativo de optimización que busca el mínimo moviéndose en la dirección opuesta al gradiente del error $E(\mathbf{w})$:

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \eta \nabla E(\mathbf{w}^{(t)})$$

La *tasa de aprendizaje* η controla el tamaño de los pasos. El *descenso del gradiente estocástico* (SGD) utiliza el gradiente calculado sobre un único ejemplo o un pequeño lote (mini-batch) en cada iteración, siendo computacionalmente eficiente para grandes datasets y útil para navegar paisajes de error complejos con múltiples mínimos locales, una característica esencial en el entrenamiento de modelos de Aprendizaje Profundo.

3.5 INTRODUCCIÓN A LOS MODELOS TRANSFORMER

Los modelos Transformer irrumpieron en el panorama del aprendizaje automático como una auténtica disrupción, marcando un antes y un después en el procesamiento de secuencias, especialmente en el ámbito del lenguaje natural (NLP). Su concepción representó un avance paradigmático, superando las limitaciones inherentes a las arquitecturas recurrentes (RNNs) y convolucionales (CNNs) que hasta entonces dominaban el campo. La esencia de su revolución reside en dos pilares fundamentales: su capacidad intrínseca para el **procesamiento en paralelo** y la introducción de un mecanismo de atención sumamente potente y flexible. Mientras que las RNNs procesaban la información de manera secuencial, limitando la eficiencia y la capacidad de capturar dependencias a largo alcance, los Transformers abordan la secuencia de entrada como un todo, operando simultáneamente sobre todos sus elementos. Este enfoque, orquestado por el mecanismo de atención, dota al modelo de la habilidad de discernir y ponderar la importancia relativa de cada parte de la entrada con respecto a las demás. En lugar de procesar la información paso a paso, el Transformer evalúa la relevancia contextual de cada elemento, optimizando el flujo de información y, en última instancia, potenciando significativamente su capacidad de aprendizaje y comprensión. Este capítulo se adentra en el corazón de la arquitectura Transformer, desglosando sus componentes esenciales y trazando un camino que culmina en SAITS (Self-Attention-based Imputation for Time Series), un modelo especializado que adapta la potencia del Transformer a la imputación de datos faltantes en series temporales. A través de este recorrido, demostraremos la versatilidad y el alcance de esta arquitectura, extendiéndose mucho más allá de sus orígenes en el procesamiento del lenguaje natural.

3.5.1 Mecanismo de Atención Escalonada (Scaled Dot-Product Attention)

El mecanismo de atención constituye el núcleo operativo y la innovación distintiva de los Transformers, siendo el principal artífice de su excepcional rendimiento. Dentro del espectro de mecanismos de atención existentes, la **atención escalonada por producto punto** (Scaled Dot-Product Attention) emerge como la piedra angular, el bloque constructivo esencial. Este ingenioso mecanismo habilita al modelo para discernir y cuantificar la similitud o relevancia entre diferentes posiciones dentro de la secuencia de entrada. De esta forma, determina con precisión qué fragmentos de la información entrante merecen una mayor ponderación al procesar cada elemento en particular. En términos prácticos, la atención confiere al modelo la capacidad de enfocar su "mirada" selectivamente sobre la información más crucial y pertinente en cada instante del procesamiento, emulando, en cierto modo, la atención selectiva humana.

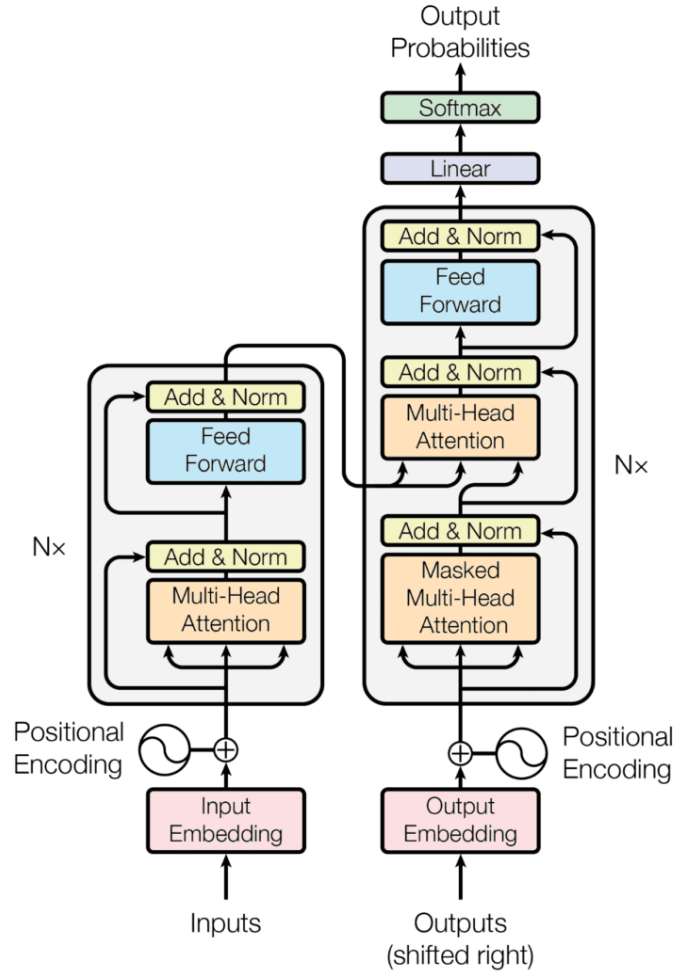


Figura 1: Diagrama conceptual de un bloque Transformer típico (Encoder y Decoder).

3.5.1.1 Matrices de Consulta, Clave y Valor (Query, Key, Value)

La orquestación de la atención escalonada se basa en la interacción estratégica de tres matrices fundamentales: **Consulta (Query)**, **Clave (Key)** y **Valor (Value)**.

La esencia matemática del mecanismo de atención escalonada se encapsula en la Ecuación 3:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (3)$$

donde:

- $Q \in \mathbb{R}^{n \times d_k}$ representa la matriz de Consulta, con n siendo la longitud de la secuencia y d_k la dimensión de las consultas y claves.

- $K \in \mathbb{R}^{m \times d_k}$ es la matriz de Clave, con m siendo la longitud de la secuencia (que puede ser diferente de n en algunos casos, como la atención Encoder-Decoder) y d_k la dimensión de las claves.
- $V \in \mathbb{R}^{m \times d_v}$ denota la matriz de Valor, con m siendo la longitud de la secuencia de valores y d_v la dimensión de los valores.
- d_k es el factor de escala, específicamente la raíz cuadrada de la dimensión de las claves (d_k), que se utiliza para mitigar el problema del desvanecimiento del gradiente, especialmente con dimensiones elevadas.
- softmax es la función softmax, encargada de normalizar las puntuaciones de similitud, transformándolas en una distribución de probabilidad que representa los pesos de atención.

s

Estas matrices, representadas convencionalmente como Q , K y V respectivamente, no son entidades independientes, sino que se derivan de la secuencia de entrada original a través de proyecciones lineales individuales y dedicadas. Para comprender mejor su función, podemos recurrir a una analogía conceptual:

- **Consulta (Q):** La matriz de Consulta (Q) personifica la pregunta o la necesidad de información del elemento que se está procesando en un momento dado. Cada fila de Q representa un elemento de la secuencia de entrada y encapsula la consulta o la búsqueda de información relevante en el resto de la secuencia que emana de dicho elemento. Imaginemos que cada palabra en una frase es una consulta buscando entender su contexto.
- **Clave (K):** La matriz de Clave (K) actúa como un catálogo de claves o etiquetas descriptivas que identifican las diferentes partes de la secuencia de entrada. Al igual que Q , cada fila de K corresponde a un elemento de la secuencia de entrada, pero en este caso, representa la clave que describe la información contenida en ese elemento y que podría ser relevante para responder a las consultas. Siguiendo con la analogía, cada palabra en la frase también proporciona una clave que describe su propio significado y rol en la oración.
- **Valor (V):** La matriz de Valor (V) almacena la información concreta o el valor asociado a cada clave. De nuevo, cada fila de V se corresponde con un elemento de la secuencia de entrada y contiene la información que realmente se utilizará para construir la salida final, una vez ponderada por la atención. En nuestra analogía, la matriz de valor contendría la representación vectorial del significado de cada palabra, lista para ser combinada y ponderada para formar una representación contextualizada de la frase.

El proceso computacional que subyace a la atención escalonada se desarrolla en los siguientes pasos:

1. **Cálculo de Similitud (Producto Punto):** Inicialmente, se evalúa la similitud entre cada consulta y cada clave mediante el **producto punto matricial** QK^T . Este

producto punto cuantifica el grado de correspondencia o alineamiento entre las consultas y las claves, generando una matriz de **puntuaciones de similitud**.

2. **Escalado:** Las puntuaciones de similitud obtenidas se someten a un proceso de **escalado**, dividiéndolas por $\sqrt{d_k}$. Este paso de escalado es de vital importancia para la **estabilidad del entrenamiento**, ya que previene que las puntuaciones se disparen a valores excesivamente grandes, lo cual podría conducir a **gradientes extremadamente pequeños** tras la aplicación de la función softmax, dificultando el **aprendizaje efectivo**.
3. **Normalización (Softmax):** A continuación, la **función softmax** entra en juego para **normalizar** las puntuaciones escaladas. La softmax transforma estas puntuaciones en **pesos de atención probabilísticos**, que oscilan entre 0 y 1 y suman 1 para cada consulta. Estos **pesos de atención** reflejan la **importancia relativa** de cada valor con respecto a cada consulta, indicando en qué medida cada valor debe contribuir a la representación final.
4. **Ponderación y Agregación (Ponderación por Valores):** Finalmente, los pesos de atención normalizados se utilizan para **ponderar** la matriz de Valor V. Esta ponderación se materializa multiplicando los pesos de atención por los valores correspondientes y agregando los resultados mediante una **suma ponderada**. La **salida resultante** de este proceso es una matriz que representa la secuencia de entrada, pero ahora **enriquecida con información contextualizada**, donde cada elemento se ha refinado y ajustado en función de su relevancia con respecto a los demás elementos de la secuencia, guiado por el mecanismo de atención.

3.5.2 Estructura de Capas en un Transformer

Un Transformer típico se compone de varias capas de codificación (**Encoder**) y decodificación (**Decoder**). Estas capas se apilan secuencialmente, creando una arquitectura profunda capaz de aprender representaciones complejas de los datos. Cada capa, tanto en el Encoder como en el Decoder, está construida con un conjunto específico de subcapas interconectadas, diseñadas para extraer información relevante y transformar los datos de entrada. Las subcapas fundamentales incluyen la **atención multi-cabeza** (Multi-Head Attention), **redes neuronales feed-forward**, **normalización de capas** (Layer Normalization) y **conexiones residuales**. A continuación, se detalla la estructura y función de cada componente.

3.5.2.1 Capas del Encoder

Cada capa del Encoder está diseñada para procesar la secuencia de entrada y extraer características relevantes. Típicamente, una capa de Encoder consta de las siguientes subcapas principales:

1. **Atención Multi-Cabeza (Multi-Head Attention):** Esta es la subcapa central del Transformer. Permite al modelo ponderar la importancia de diferentes partes de la secuencia de entrada al procesar cada posición. En lugar de una única atención, la atención multi-cabeza utiliza múltiples "cabezas" de atención en paralelo. Cada cabeza aprende a atender a diferentes aspectos de la información, capturando una variedad más rica de relaciones y dependencias dentro de la secuencia. La salida de todas las cabezas se concatena y se proyecta linealmente para producir la salida final de esta subcapa.
2. **Añadir y Normalizar (Add & Norm) - Conexión Residual y Normalización de Capas:** Después de la subcapa de atención multi-cabeza, se aplica una **conexión residual**. Esto significa que la entrada original a la subcapa de atención se suma a la salida de la atención multi-cabeza. Esta conexión residual ayuda a mitigar el problema del desvanecimiento del gradiente en redes profundas, facilitando el entrenamiento de modelos más profundos. A continuación, se aplica la **normalización de capas** (Layer Normalization) (Ba et al., 2016). La normalización de capas ayuda a estabilizar el aprendizaje normalizando las salidas de las subcapas, lo que acelera el entrenamiento y mejora la generalización.
3. **Red Neuronal Feed-Forward (Feed-Forward Network):** Tras la subcapa de atención y la normalización, la salida se pasa a través de una **red neuronal feed-forward**. Esta red es aplicada de forma idéntica e independiente a cada posición en la secuencia. Consiste típicamente en dos capas lineales con una función de activación no lineal (como ReLU) entre ellas. La red feed-forward ayuda a procesar la información aprendida por la atención, proporcionando no linealidad y permitiendo al modelo aprender transformaciones más complejas.
4. **Añadir y Normalizar (Add & Norm) - Conexión Residual y Normalización de Capas:** Similar al paso anterior, se aplica otra conexión residual y normalización de capas después de la red neuronal feed-forward. La entrada a la red feed-forward se suma a su salida, y el resultado se normaliza.

3.5.2.2 Capas del Decoder

Las capas del Decoder tienen la función de generar la secuencia de salida, utilizando la información codificada por el Encoder. Una capa de Decoder es similar a una capa de Encoder, pero incluye una subcapa de atención adicional para interactuar con la salida del Encoder. Las subcapas típicas de una capa de Decoder son:

1. **Atención Multi-Cabeza Enmascarada (Masked Multi-Head Attention):** Similar a la atención multi-cabeza del Encoder, pero con una modificación crucial: el **enmascaramiento**. En el contexto del Decoder, especialmente en tareas de generación secuencial, es importante que al predecir la salida en una posición dada, el modelo solo pueda atender a las posiciones *anteriores* en la secuencia de salida (ya generada) y no a las posiciones futuras. El enmascaramiento se im-

plementa para evitar que el modelo "vea" la respuesta durante el entrenamiento, asegurando que aprenda a generar la secuencia de forma autoregresiva.

2. **Añadir y Normalizar (Add & Norm):** Conexión residual y normalización de capas aplicada después de la atención multi-cabeza enmascarada, de la misma manera que en el Encoder.
3. **Atención Encoder-Decoder (Encoder-Decoder Attention o Multi-Head Attention):** Esta subcapa es específica del Decoder y es fundamental para la interacción entre el Encoder y el Decoder. En esta subcapa, el Decoder atiende a la salida del *Encoder*. Las "queries" provienen de la salida de la subcapa de atención enmascarada del Decoder, mientras que las "keys" "values" provienen de la salida del *Encoder*. Esto permite al Decoder enfocar su atención en las partes relevantes de la secuencia de entrada codificada por el Encoder al generar cada elemento de la secuencia de salida.
4. **Añadir y Normalizar (Add & Norm):** Conexión residual y normalización de capas aplicada después de la atención Encoder-Decoder.
5. **Red Neuronal Feed-Forward (Feed-Forward Network):** Idéntica a la red neuronal feed-forward utilizada en las capas del Encoder.
6. **Añadir y Normalizar (Add & Norm):** Conexión residual y normalización de capas aplicada después de la red neuronal feed-forward.

3.5.2.3 Diagrama Conceptual de un Bloque Transformer

Un diagrama de un bloque Transformer típico mostraría:

- **Para un bloque Encoder:**
 - Un bloque etiquetado Entrada que representa la secuencia de entrada (o la salida de la capa Encoder anterior).
 - Una flecha que lleva a un bloque etiquetado Multi-Head Attention.
 - Una conexión residual que salta la subcapa "Multi-Head Attention" y se suma a la salida de esta subcapa.
 - Un bloque etiquetado Add & Norm después de la suma de la conexión residual.
 - Una flecha desde Add & Norm a un bloque etiquetado Feed-Forward Network.
 - Otra conexión residual que salta la subcapa Feed-Forward Network y se suma a la salida de esta subcapa.
 - Un bloque etiquetado Add & Norm después de la segunda suma de la conexión residual, representando la salida de la capa Encoder.
- **Para un bloque Decoder:**
 - Un bloque etiquetado Entrada que representa la secuencia de entrada al Decoder (o la salida de la capa Decoder anterior).
 - Una flecha que lleva a un bloque etiquetado Masked Multi-Head Attention.

- Una conexión residual y Add & Norm después de la atención enmascarada.
- Una flecha a un bloque etiquetado Encoder-Decoder Attention (o simplemente Multi-Head Attention si se entiende el contexto). Este bloque también recibiría una conexión de la salida del Encoder (en un diagrama para la arquitectura completa).
- Una conexión residual y Add & Norm después de la atención Encoder-Decoder.
- Una flecha a un bloque etiquetado Feed-Forward Network.
- Una conexión residual y Add & Norm después de la red feed-forward, representando la salida de la capa Decoder.

En resumen, la estructura de capas de un Transformer, con sus componentes de atención multi-cabeza, redes feed-forward, normalización de capas y conexiones residuales, proporciona un marco robusto y eficiente para el modelado de secuencias complejas, tanto en tareas de codificación como de decodificación. La atención multi-cabeza es el mecanismo central que permite al modelo capturar dependencias ricas, mientras que las otras subcapas facilitan el entrenamiento y mejoran la capacidad de aprendizaje de la red.

3.6 ENTRENAMIENTO DEL TRANSFORMER

El entrenamiento de un Transformer [Vaswani et al. \(2017\)](#), como modelo predictivo f_θ , se basa en minimizar una función de pérdida $L(\theta)$ utilizando descenso de gradiente.

- **Parámetros θ :** Incluyen matrices de pesos $\mathbf{W}$ y vectores de sesgo $\mathbf{b}$ (??).
- **Función de Pérdida $L(\theta)$:** Mide el error entre la salida $\hat{\mathbf{Y}} = f_\theta(\mathbf{X})$ y la referencia $\mathbf{Y}$ (ej. MSE usando normas matriciales).
- **Descenso de Gradiente:** Actualiza $\theta^{(t+1)} = \theta^{(t)} - \eta \nabla L(\theta^{(t)})$.
- **Gradiente $\nabla L(\theta)$:** Es la colección de derivadas parciales $\frac{\partial L}{\partial w_{ij}}, \frac{\partial L}{\partial b_i}$ para todos los parámetros. Se calcula mediante **Backpropagation**, que aplica eficientemente la Regla de la Cadena (??) a través de las operaciones del modelo (producto matricial, suma, Softmax, etc.).

El proceso requiere herramientas del cálculo diferencial multivariable (??) y álgebra lineal (??).

Algorithm 1 Algoritmo de Retropropagación

```

1: Entrada: Lote de datos de entrada  $X = A^0$ , Lote de datos objetivo  $Y$ , Parámetros
   actuales  $\theta = \{W^l, b^l\}_{l=1}^{N_L}$ , Funciones de activación  $\{g^l\}_{l=1}^{N_L}$ , Función de Pérdida  $\mathcal{L}$ .
2: Salida: Gradiente  $\nabla_{\theta} \mathcal{L} = \{\nabla_{W^l} \mathcal{L}, \nabla_{b^l} \mathcal{L}\}_{l=1}^{N_L}$ .

   // — Fase 1: Pase Hacia Adelante (Forward Pass) —
3: for  $l = 1$  to  $N_L$  do                                     ▷ Iterar sobre las capas hacia adelante
4:   Calcular pre-activación:  $Z^l \leftarrow f_{\text{linear}}(A^{l-1}, W^l, b^l)$            ▷ Ej:  $W^l A^{l-1} + b^l$ 
5:   Calcular activación:  $A^l \leftarrow g^l(Z^l)$ 
6:   Almacenar  $A^{l-1}$  y  $Z^l$  para la fase hacia atrás.
7: Calcular la predicción final:  $\hat{Y} \leftarrow A^{N_L}$ 
8: Calcular la pérdida:  $L \leftarrow \mathcal{L}(\hat{Y}, Y)$ 

   // — Fase 2: Pase Hacia Atrás (Backward Pass) —
9: Inicializar el error retropropagado (gradiente respecto a la última pre-activación):
10:  $\delta^{N_L} \leftarrow \frac{\partial \mathcal{L}}{\partial \hat{Y}} \odot (g^{N_L})'(Z^{N_L})$            ▷ Aplicar regla de la cadena en la última capa
11: for  $l = N_L$  down to  $1$  do                                   ▷ Iterar sobre las capas hacia atrás
12:   // Calcular gradientes para los parámetros de la capa  $l$ 
13:   Recuperar  $A^{l-1}$  (almacenada en el forward pass).
14:    $\nabla_{W^l} \mathcal{L} \leftarrow \frac{\partial \mathcal{L}}{\partial W^l}$                                ▷ Ej:  $\delta^l (A^{l-1})^T$  para capa lineal
15:    $\nabla_{b^l} \mathcal{L} \leftarrow \frac{\partial \mathcal{L}}{\partial b^l}$                                ▷ Ej: suma de  $\delta^l$  sobre el batch para capa lineal
16:   // Propagar el error a la capa anterior (si no es la primera capa)
17:   if  $l > 1$  then
18:     Calcular gradiente respecto a la activación de la capa anterior:
19:      $\nabla_{A^{l-1}} \mathcal{L} \leftarrow (\frac{\partial Z^l}{\partial A^{l-1}})^T \delta^l$            ▷ Ej:  $(W^l)^T \delta^l$  para capa lineal
20:     Recuperar  $Z^{l-1}$  (almacenada en el forward pass).
21:     Calcular el error para la capa anterior:
22:      $\delta^{l-1} \leftarrow \nabla_{A^{l-1}} \mathcal{L} \odot (g^{l-1})'(Z^{l-1})$ 
23: Devolver los gradientes acumulados  $\{\nabla_{W^l} \mathcal{L}, \nabla_{b^l} \mathcal{L}\}_{l=1}^{N_L}$ 

```

NOTAS SOBRE EL ALGORITMO:

- Las expresiones exactas para $\frac{\partial \mathcal{L}}{\partial W^l}$, $\frac{\partial \mathcal{L}}{\partial b^l}$, $\frac{\partial Z^l}{\partial A^{l-1}}$ y $\frac{\partial \mathcal{L}}{\partial \hat{Y}}$ dependen de la forma específica de las capas lineales (o convolucionales, de atención, etc.) y de la función de pérdida utilizada. El algoritmo presenta la estructura general.
- δ^l representa $\frac{\partial \mathcal{L}}{\partial Z^l}$, que es el error retropropagado justo antes de la función de activación de la capa l .
- $(g^l)'(Z^l)$ denota la derivada de la función de activación g^l evaluada en la pre-activación Z^l .
- La eficiencia del algoritmo reside en el cálculo y almacenamiento de los valores intermedios durante el pase hacia adelante y su reutilización durante el pase hacia atrás para aplicar la regla de la cadena de forma secuencial.

- En implementaciones reales (PyTorch, TensorFlow), este proceso es automatizado por el sistema de diferenciación automática (autograd), pero sigue esta lógica subyacente.
- **Costo Computacional de la Atención:** El cálculo de la atención, especialmente en secuencias largas, puede ser computacionalmente costoso tanto en el paso forward como en el backward. La complejidad de la atención escalonada es cuadrática con respecto a la longitud de la secuencia ($O(L^2)$, donde L es la longitud de la secuencia). Esto puede ser un cuello de botella durante el entrenamiento, especialmente para secuencias muy largas. Técnicas como la atención dispersa (sparse attention) o la atención lineal se han propuesto para mitigar este problema.
- **Estabilidad del Gradiente:** Las conexiones residuales y la normalización de capas en los Transformers ayudan a mejorar la estabilidad del gradiente durante la retropropagación, facilitando el entrenamiento de redes profundas y reduciendo problemas como el desvanecimiento o la explosión del gradiente.
- **Paralelización:** Aunque el mecanismo de atención permite la paralelización en el paso forward, la retropropagación también se puede paralelizar en gran medida, especialmente en hardware como GPUs, lo que acelera significativamente el entrenamiento de los Transformers.

3.7 SELF-ATTENTION-BASED IMPUTATION FOR TIME SERIES (SAITS)

SAITS (Self-Attention-based Imputation for Time Series) es un modelo Transformer específicamente diseñado para la imputación de datos faltantes en series temporales. Aprovecha el mecanismo de auto-atención para capturar dependencias temporales y espaciales en los datos de series temporales, lo que permite una imputación más precisa en comparación con los métodos tradicionales. A diferencia de los modelos de imputación clásicos que a menudo se basan en suposiciones simplificadas sobre la estructura temporal de los datos, SAITS aprende directamente estas dependencias de los propios datos, ofreciendo una solución más flexible y robusta.

3.7.1 Arquitectura del Modelo SAITS

SAITS adapta la arquitectura Transformer estándar para la tarea de imputación de series temporales, principalmente utilizando una estructura de **Encoder Transformer**. Dado que el objetivo principal es la imputación y no la generación secuencial, la arquitectura Decoder no es estrictamente necesaria en este contexto, aunque variaciones podrían incluirla. Las adaptaciones clave en la arquitectura de SAITS para la imputación de series temporales son:

- **Entrada de Series Temporales:** SAITS toma como entrada series temporales con posibles valores faltantes. Típicamente, la serie temporal se representa como una secuencia de vectores, donde cada vector representa un punto de tiempo y contiene las mediciones de diferentes variables en ese instante. Para manejar los valores faltantes, se pueden emplear **máscaras de entrada** que indican las posiciones donde los datos son desconocidos. Estas máscaras pueden ser utilizadas por el modelo para enfocar la atención en las partes observadas de la serie temporal durante la imputación.
- **Encoder Transformer para Imputación:** SAITS utiliza un stack de capas Encoder Transformer. Cada capa Encoder, como se describió anteriormente, contiene subcapas de atención multi-cabeza, redes feed-forward, normalización de capas y conexiones residuales. Estas capas procesan la serie temporal de entrada para aprender representaciones latentes que capturan las dependencias temporales y las relaciones entre las variables. La **auto-atención** dentro del Encoder permite que cada punto de tiempo en la serie temporal atienda a todos los demás puntos, ponderando su importancia relativa para la imputación en la posición actual.
- **Codificación Posicional Adaptada (Opcional):** Si bien la codificación posicional es crucial en NLP para indicar el orden secuencial de las palabras, su uso en series temporales puede ser adaptado. En algunos casos, la posición temporal relativa ya está implícita en la secuencia de entrada. Sin embargo, se pueden utilizar codificaciones posicionales para explicitar la posición temporal y potencialmente codificar información adicional como el tiempo transcurrido desde el inicio de la serie o patrones temporales cíclicos (e.g., día de la semana, mes del año). La decisión de incluir o adaptar la codificación posicional puede depender de las características específicas del conjunto de datos de series temporales.
- **Capa de Salida para Imputación:** La salida del Encoder Transformer, tras pasar por las capas Encoder, se utiliza para generar los valores imputados. Típicamente, esto se realiza mediante una **capa lineal** final que proyecta la representación latente de cada punto de tiempo a la dimensión del espacio de características original. Esta capa lineal aprende a mapear las representaciones aprendidas por el Transformer a los valores imputados para cada variable en cada punto de tiempo donde falta información.

3.7.2 Función de Pérdida para SAITS

La función de pérdida comúnmente utilizada para entrenar SAITS en tareas de imputación es el **Error Cuadrático Medio (MSE)** o variantes del mismo, aplicadas **solo a las posiciones donde los datos estaban originalmente faltantes**. Esto se debe a que el objetivo es minimizar el error en la predicción de los valores faltantes, sin penalizar al modelo por los valores ya observados. Formalmente, si Y es la serie temporal original (con valores faltantes), $\hat{Y}$ es la serie temporal imputada por SAITS y M es

una matriz máscara binaria donde $M_{ij} = 1$ si el valor Y_{ij} estaba faltante y $M_{ij} = 0$ si estaba observado, la función de pérdida MSE puede definirse como:

$$\mathcal{L}_{MSE} = \frac{1}{\sum_{i,j} M_{ij}} \sum_{i,j} M_{ij} (Y_{ij} - \hat{Y}_{ij})^2 \quad (4)$$

Esta función de pérdida asegura que el modelo se centre en aprender a imputar los valores faltantes con precisión, ignorando el error en las posiciones donde ya se disponía de información. Otras funciones de pérdida robustas, como el Mean Absolute Error (MAE) o funciones de pérdida basadas en cuantiles, también podrían ser consideradas dependiendo de las características de los datos y los objetivos específicos de la imputación.

3.7.3 Ventajas de Self-Attention para Imputación de Series Temporales

El uso del mecanismo de auto-atención en SAITS ofrece varias ventajas significativas para la imputación de series temporales en comparación con métodos tradicionales:

- **Modelado de Dependencias a Largo Plazo:** La auto-atención permite a SAITS capturar dependencias temporales a largo plazo de manera efectiva. A diferencia de métodos como ARIMA o RNNs que pueden tener dificultades para recordar información a través de largos periodos de tiempo, la atención permite a SAITS acceder directamente a cualquier punto en la serie temporal al imputar un valor, sin importar la distancia temporal. Esto es crucial para series temporales con patrones complejos y dependencias que se extienden a lo largo del tiempo.
- **Captura de Dependencias Espaciales y Temporales Conjuntas:** En series temporales multivariadas, donde se miden múltiples variables a lo largo del tiempo, la auto-atención puede capturar tanto las dependencias temporales dentro de cada variable como las dependencias espaciales (correlaciones) entre diferentes variables en diferentes puntos de tiempo. El modelo aprende a imputar un valor faltante considerando la información de todas las variables en todos los puntos de tiempo relevantes.
- **Flexibilidad y Adaptabilidad:** La arquitectura Transformer es inherentemente flexible y adaptable a diferentes tipos de series temporales y patrones de datos faltantes. SAITS no requiere suposiciones fuertes sobre la estacionariedad de la serie temporal o la naturaleza de las dependencias temporales. Puede aprender patrones complejos directamente de los datos y adaptarse a diferentes tipos de series temporales sin necesidad de ingeniería de características manual extensiva.
- **Robustez ante Datos Irregulares y Valores Atípicos:** La atención puede ayudar a SAITS a ser más robusto ante datos irregulares y valores atípicos en las series temporales. Al ponderar la importancia de diferentes partes de la secuencia, la

atención puede dar menos peso a los valores atípicos o a las regiones ruidosas de la serie temporal durante la imputación, enfocándose en la información más consistente y relevante.

En resumen, SAITS representa un avance significativo en la imputación de series temporales al aplicar la poderosa arquitectura Transformer y el mecanismo de auto-atención. Su capacidad para modelar dependencias complejas, su flexibilidad y robustez lo convierten en una herramienta valiosa para el análisis de datos de series temporales con valores faltantes en diversos dominios.

3.7.4 Imputación de Datos Faltantes

Los conjuntos de datos del mundo real frecuentemente contienen valores faltantes (*missing data*). Abordar incorrectamente estos datos puede introducir sesgos significativos o reducir drásticamente la cantidad de datos utilizables (Bishop, 2006, Capítulo 8). La **imputación** es el proceso de rellenar estos valores ausentes con valores estimados, restaurando así la completitud del conjunto de datos y mitigando los efectos negativos de la ausencia de información.

Las estrategias de imputación para series temporales se pueden clasificar según la información que utilizan y los supuestos subyacentes. Adoptaremos una taxonomía centrada en enfoques locales, estacionarios y predictivos:

- **Métodos de Imputación con Información Local:** Estos métodos estiman los valores faltantes basándose principalmente en los puntos de datos cercanos temporalmente. Explotan la autocorrelación temporal, asumiendo que las observaciones cercanas en el tiempo tienden a tener valores similares.
 - *Forward Fill (ffill)* o Última Observación Llevada Adelante (LOCF): Rellena un valor faltante con el último valor conocido válido que lo precede en la secuencia. Simple y útil si los valores tienden a persistir.
 - *Backward Fill (bfill)* o Próxima Observación Llevada Atrás (NOCB): Rellena utilizando el siguiente valor conocido válido en la secuencia.
 - *Interpolación Lineal:* Estima valores asumiendo una relación lineal entre los puntos conocidos inmediatamente anterior y posterior al hueco. Es eficiente pero limitada para capturar patrones no lineales o cambios abruptos.
 - *Interpolación Spline:* Ajusta polinomios por partes (generalmente cúbicos) a los datos existentes para crear una curva suave que pasa por los puntos conocidos, imputando los valores faltantes a lo largo de esta curva (Bishop, 2006, Sección 3.1). Captura mejor patrones no lineales que la interpolación lineal, pero puede ser computacionalmente más intensiva.

Los métodos locales son efectivos para rellenar huecos cortos, pero su rendimiento decrece con lagunas más grandes y pueden ser sensibles al ruido.

- **Métodos de Imputación Estacionarios:** Estos métodos se basan en el supuesto de que las propiedades estadísticas de la serie temporal (o segmentos de ella),

como la media o la varianza, son constantes a lo largo del tiempo. Utilizan estadísticas globales o segmentadas de los datos observados.

- *Media*: Reemplaza los valores faltantes con la media aritmética de toda la serie observada (o de un segmento relevante). Preserva la media global pero reduce la varianza y distorsiona las correlaciones y la estructura temporal (Bishop, 2006, Capítulo 1).
- *Mediana*: Similar a la imputación por media, pero utiliza la mediana. Es más robusta a valores atípicos en distribuciones sesgadas, pero comparte las limitaciones de la imputación por media respecto a la estructura temporal y la varianza.
- *Moda*: Reemplaza los valores faltantes con el valor más frecuente. Se utiliza principalmente para datos categóricos o discretos.

Estos métodos son simples pero fallan en capturar tendencias, estacionalidad y dependencias temporales, y son sensibles a violaciones del supuesto de estacionariedad.

- **Métodos de Imputación Predictivos**: Utilizan modelos estadísticos o de aprendizaje automático para predecir los valores faltantes, aprendiendo patrones de los datos observados. Pueden capturar relaciones más complejas Abu-Mostafa et al. (2012).
 - *Modelos de Regresión*: Se ajusta un modelo (lineal o no lineal) usando el tiempo u otras variables (en caso multivariante) como predictores para estimar los valores faltantes (Bishop, 2006, Capítulo 3).
 - *Modelos de Aprendizaje Automático (Convencionales)*: Algoritmos como K-Vecinos Más Cercanos (KNN) o Bosques Aleatorios (Random Forests) pueden usarse. KNN imputa basándose en un promedio de los valores de los K vecinos más similares (Bishop, 2006, Sección 2.5.2).
 - *Modelos de Aprendizaje Profundo*: Arquitecturas como Redes Neuronales Recurrentes (RNNs, incluyendo LSTM y GRU) o Transformers son especialmente potentes para series temporales, ya que pueden capturar dependencias temporales complejas y de largo alcance (Goodfellow et al., 2016, Capítulos 10 y 12).

Estos métodos son generalmente más potentes y flexibles, capaces de manejar patrones complejos y datos multivariantes. Sin embargo, suelen requerir más datos, son computacionalmente más costosos y conllevan un riesgo de sobreajuste.

La elección del método de imputación adecuado depende crucialmente de las características de los datos (estacionariedad, linealidad, presencia de outliers, estacionalidad), del patrón y porcentaje de valores faltantes, y de los recursos computacionales disponibles Goodfellow et al. (2016). Un manejo y elección cuidadosos son fundamentales para la validez de cualquier análisis o modelo posterior construido sobre los datos imputados.

3.8 MÉTRICAS DE EVALUACIÓN PARA IMPUTACIÓN

Para cuantificar el rendimiento de los métodos de imputación, utilizamos métricas de error estándar que comparan los valores imputados $\hat{\mathcal{X}}^{\text{test}}$ con los valores verdaderos $\mathcal{X}_{\text{orig}}^{\text{test}}$ *únicamente* en las posiciones indicadas por la máscara $\mathbf{M}_{\text{artificial}}^{\text{test}}$. Estas cantidades se representan como tensores (generalizaciones de matrices, ver ??).

Sean:

- $\hat{\mathcal{X}} = \hat{\mathcal{X}}^{\text{test}} \in \mathbb{R}^{N \times T \times D}$: Tensor de valores imputados (test).
- $\mathbf{X}_{\text{orig}} = \mathcal{X}_{\text{orig}}^{\text{test}} \in \mathbb{R}^{N \times T \times D}$: Tensor de valores verdaderos originales (test).
- $\mathbf{M}_{\text{ind}} = \mathbf{M}_{\text{artificial}}^{\text{test}} \in \{0, 1\}^{N \times T \times D}$: Máscara binaria que indica las posiciones imputadas (1 si fue imputada/artificialmente eliminada, 0 si no)

Las métricas se definen como:

Definición 40 (Error Absoluto Medio (MAE)).

$$\text{MAE}(\hat{\mathcal{X}}, \mathbf{X}_{\text{orig}}, \mathbf{M}_{\text{ind}}) = \frac{|||\hat{\mathcal{X}} - \mathbf{X}_{\text{orig}}| \odot \mathbf{M}_{\text{ind}}||_1}{||\mathbf{M}_{\text{ind}}||_1} \quad (5)$$

Aquí, $|\hat{\mathcal{X}} - \mathbf{X}_{\text{orig}}|$ calcula la diferencia absoluta elemento a elemento. El término $\odot \mathbf{M}_{\text{ind}}$ selecciona solo las posiciones indicadas por la máscara mediante el producto de Hadamard (??), y $|| \cdot ||_1$ suma estos errores absolutos. El denominador $||\mathbf{M}_{\text{ind}}||_1$ normaliza por el número de puntos imputados.

Definición 41 (Raíz del Error Cuadrático Medio (RMSE)).

$$\text{RMSE}(\hat{\mathcal{X}}, \mathbf{X}_{\text{orig}}, \mathbf{M}_{\text{ind}}) = \sqrt{\frac{|||(\hat{\mathcal{X}} - \mathbf{X}_{\text{orig}})^{\odot 2} \odot \mathbf{M}_{\text{ind}}||_1}{||\mathbf{M}_{\text{ind}}||_1}} \quad (6)$$

Similarmente, $(\hat{\mathcal{X}} - \mathbf{X}_{\text{orig}})^{\odot 2}$ calcula el error cuadrático elemento a elemento (usando el producto de Hadamard consigo mismo, ??). El término $\odot \mathbf{M}_{\text{ind}}$ lo enmascara (nuevamente con ??), y $|| \cdot ||_1$ suma los errores cuadráticos. Se divide por el número de puntos y se toma la raíz cuadrada.

Definición 42 (Error Relativo Medio (MRE)).

$$\text{MRE}(\hat{\mathcal{X}}, \mathbf{X}_{\text{orig}}, \mathbf{M}_{\text{ind}}) = \frac{|||\hat{\mathcal{X}} - \mathbf{X}_{\text{orig}}| \odot \mathbf{M}_{\text{ind}}||_1}{|||\mathbf{X}_{\text{orig}}| \odot \mathbf{M}_{\text{ind}}||_1 + \epsilon} \quad (7)$$

El numerador es la suma de los errores absolutos enmascarados (usando $\odot \mathbf{M}_{\text{ind}}$, ??). El denominador es la suma de los valores absolutos verdaderos originales en las posiciones enmascaradas, aplicando también el producto de Hadamard (??), con una pequeña constante $\epsilon > 0$ para evitar la división por cero.

Parte III

BLOQUE III: METODOLOGÍA Y CASO DE ESTUDIO: IMPUTACIÓN DE DATOS

IMPUTACIÓN DE DATOS DE TRÁFICO Y CONTAMINACIÓN

4.1 CONTEXTO DEL PROBLEMA Y MOTIVACIÓN

El presente estudio se enmarca en el ámbito de las *Smart Villages* o pueblos inteligentes, aplicando tecnologías del Internet de las Cosas (IoT) para monitorizar y analizar dinámicas ambientales y de movilidad en entornos rurales. Específicamente, se utiliza la infraestructura desplegada en el proyecto Smart Poqueira ([Universidad de Granada, 2023](#)), localizado en la Alpujarra Granadina, una zona de alto valor ecológico y turístico. La infraestructura IoT consta, entre otros sensores, de cámaras con reconocimiento de matrículas (LPR - License Plate Recognition) estratégicamente ubicadas en los accesos a los municipios de Pampaneira, Bubi3n y Capileira (Figura 2), y una unidad móvil (camión) equipada con sensores para medir diversos contaminantes atmosféricos y variables meteorológicas.



Figura 2: Distribuci3n geogr3fica de las c3maras LPR (License Plate Recognition) en los municipios de Pampaneira, Capileira y Bubi3n, utilizadas para la monitorizaci3n del tr3fico vehicular en el proyecto Smart Poqueira. Se indican las ubicaciones espec3ficas y la direcci3n de monitorizaci3n.

El objetivo general es comprender las relaciones entre los patrones de tráfico vehicular y los niveles de contaminación atmosférica en esta región. Sin embargo, surge un desafío fundamental inherente a la naturaleza móvil de la unidad de medición de contaminantes: el camión de meteorología y contaminación opera en ubicaciones específicas (Pampaneira-Bubión) solo durante periodos de tiempo definidos:

- **Capileira:** 16/11/2022 - 15/01/2023 (Estos datos no se tratan en este análisis específico de imputación).
- **Pampaneira-Bubión (Cementerio):** 17/01/2023 - 14/03/2023.
- **Pampaneira-Bubión (Cementerio):** 06/06/2023 - 27/06/2023.

Esta operativa, junto con posibles fallos intermitentes de los sensores o cámaras, genera **datos faltantes** sistemáticos en las series temporales combinadas de tráfico y contaminación. La ausencia de estas mediciones impide un análisis continuo y completo de la relación tráfico-contaminación.

Por lo tanto, la **motivación principal** de este trabajo es abordar el problema de los datos faltantes mediante la aplicación y evaluación de diferentes técnicas de **imputación**. El objetivo es estimar los valores ausentes de manera robusta y precisa, aprovechando la información contenida en los datos existentes, para así poder realizar análisis posteriores más completos y fiables.

4.2 CONJUNTOS DE DATOS

Para llevar a cabo este estudio, se utilizan dos conjuntos de datos principales, derivados de la infraestructura IoT descrita.

4.2.1 *Dataset de Tráfico General*

Este dataset contiene los registros agregados por hora del tráfico vehicular detectado por las 4 cámaras LPR durante un periodo extenso (febrero 2022 - agosto 2023). Cada fila representa una hora y las columnas cuantifican vehículos categorizados por: localización/dirección (PAM_1, PAM_2, BUB, CAP; IN/OUT), origen geográfico (Zona_Granada, Zona_Andalucia_no_GR, etc.), y características inferidas (asientos, CO₂, etiqueta DGT). La columna nan agrupa vehículos sin información específica. Proporciona una visión continua de la movilidad.

4.2.2 *Dataset de Intersección Tráfico-Contaminación*

Este dataset fusiona temporalmente los datos de tráfico con las mediciones del camión móvil, conteniendo únicamente registros horarios de los periodos y ubicaciones don-

de el camión estuvo operativo en Pampaneira/Bubión. Es un subconjunto temporal del dataset de tráfico, enriquecido con variables ambientales.

4.2.2.1 Variables de Contaminación

- co Monóxido de Carbono:** Concentración media horaria medida con el analizador Thermo Scientific mod. 48i. Es un trazador de combustión incompleta, principalmente de vehículos de gasolina en condiciones de arranque en frío o ralentí, y también de la quema de biomasa. Unidades: miligramos por metro cúbico (mg/m^3).
- no Monóxido de Nitrógeno:** Concentración media horaria medida con el analizador Thermo Scientific mod. 42i. Es un contaminante primario emitido directamente por procesos de combustión a alta temperatura, siendo un **trazador importante del tráfico diésel**. Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).
- no2 Dióxido de Nitrógeno:** Concentración media horaria medida con el analizador Thermo Scientific mod. 42i. Se forma principalmente por la oxidación del NO en la atmósfera, aunque también se emite directamente en menor medida. Es, junto con el NO, un **trazador clave del tráfico diésel** y otros procesos de combustión. Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).
- o3 Ozono Troposférico:** Concentración media horaria medida con el analizador Thermo Scientific mod. 49i. Es un contaminante secundario, no emitido directamente, sino formado en la atmósfera por reacciones fotoquímicas entre óxidos de nitrógeno (NO_x) y compuestos orgánicos volátiles (COVs) en presencia de luz solar. Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).
- s02 Dióxido de Azufre:** Concentración media horaria medida con el analizador Thermo Scientific mod. 43i. Proviene principalmente de la combustión de combustibles fósiles que contienen azufre (carbón, fuelóleo). Su relevancia como trazador del tráfico rodado actual es limitada debido a las bajas concentraciones de azufre en gasolinas y gasóleos modernos. Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).
- pm10 Material Particulado <10 μm :** Concentración media horaria de partículas en suspensión con un diámetro aerodinámico inferior a 10 micrómetros, obtenida con el Monitor TEOM 1405 (Thermo Scientific). Incluye una mezcla compleja de partículas de diversas fuentes: combustión (tráfico, industria, calefacción), polvo resuspendido, aerosoles secundarios, fuentes naturales, etc. Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).
- ebc_tot Carbono Negro Total (Hollín):** Concentración media horaria de carbono negro total obtenida con el fotómetro de absorción Aethalometer AE31. El carbono negro es un producto de la combustión incompleta de material carbonoso

y es un componente importante del material particulado fino. Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).

ebc_ff Carbono Negro de Combustibles Fósiles: Concentración media horaria de carbono negro atribuible específicamente a la combustión de combustibles fósiles (gasolina, diésel). Se obtiene aplicando el "modelo del etalómetro" (Sandradewi et al., 2008). Es considerado un **trazador primario y específico del tráfico vehicular**. Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).

ebc_bb Carbono Negro de Quema de Biomasa: Concentración media horaria de carbono negro atribuible específicamente a la quema de biomasa (leña para calefacción, quemas agrícolas, incendios forestales). Se obtiene aplicando el "modelo del etalómetro" (Sandradewi et al., 2008). Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).

4.2.2.2 Variables Meteorológicas

temp Temperatura del Aire: Temperatura media horaria medida por el transmisor meteorológico YOUNG 92000 ResponseONE. Influye en la tasa de reacciones químicas atmosféricas y en la dispersión vertical de los contaminantes. Temperaturas altas en invierno pueden indicar condiciones anticiclónicas de estancamiento. Unidades: grados Celsius ($^{\circ}\text{C}$).

rh Humedad Relativa: Humedad relativa media horaria del aire medida por el transmisor YOUNG 92000. Afecta a la formación de aerosoles secundarios, la visibilidad y puede influir en la deposición de contaminantes. Alta humedad puede correlacionarse con condiciones de estancamiento o niebla. Unidades: porcentaje (%).

ws Velocidad del Viento: Velocidad media horaria del viento medida por el anemómetro de la torre meteorológica. Es un factor fundamental para la dispersión horizontal de los contaminantes; velocidades bajas favorecen la acumulación. Unidades: metros por segundo (m/s).

wd Dirección del Viento: Dirección media horaria desde la que sopla el viento, medida en la torre meteorológica. 0° corresponde al Norte geográfico. Es crucial para identificar el transporte de contaminantes desde posibles áreas fuente. Unidades: grados ($^{\circ}$).

pres Presión Atmosférica: Presión atmosférica media horaria a nivel de la estación de medición. Altas presiones suelen asociarse con condiciones meteorológicas estables (anticlones) que, combinadas con vientos bajos, dificultan la dispersión vertical y horizontal de los contaminantes, favoreciendo episodios de alta contaminación. Unidades: hectopascales (hPa).

4.2.2.3 Variable de Posición del Camión

truck_pos Posición del Camión Meteorológico: Variable categórica que indica la cámara LPR más cercana a la ubicación física del camión de medición durante la hora correspondiente. Los valores posibles en este dataset son 'CAP' (cuando estaba en Capileira, aunque esos datos no se usan aquí para la imputación) y 'PAM_2' (cuando estaba en la ubicación del cementerio de Pampaneira, monitoreado por la cámara PAM_2). Esta variable permite asociar las mediciones de contaminación con el flujo de tráfico específico de la cámara más próxima.

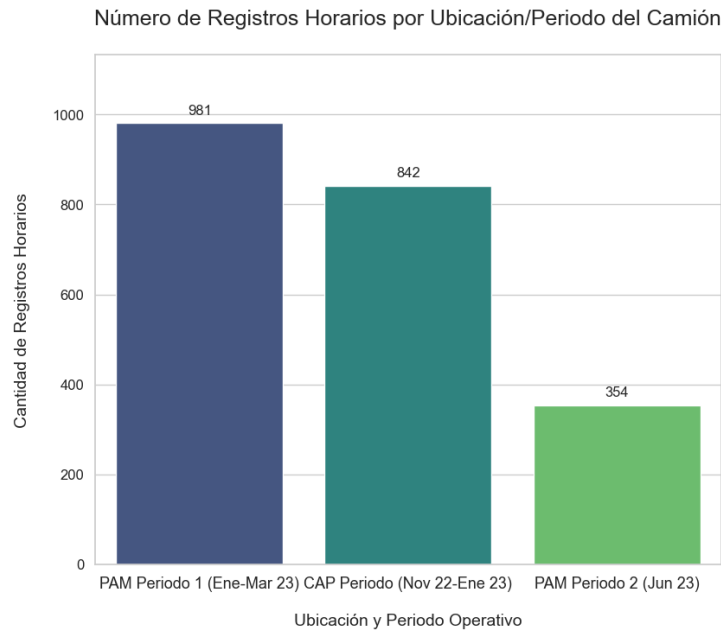


Figura 3: Distribución del número de registros horarios en el dataset de intersección (trafico_contamina_interseccion) según la ubicación y periodo operativo del camión móvil de medición (PAM Periodo 1: Ene-Mar 2023, PAM Periodo 2: Jun 2023).

Este dataset hereda también las columnas de tráfico vehicular correspondientes a la hora y cámara más cercana. Es el dataset principal para el análisis y la imputación.

4.3 ANÁLISIS EXPLORATORIO DE DATOS (EDA)

Se realizó un Análisis Exploratorio de Datos (EDA) sobre el dataset de trafico-intersección para comprender las distribuciones, identificar patrones, valores atípicos, datos faltantes y correlaciones.

4.3.1 Análisis de Valores Faltantes

El análisis del porcentaje de valores faltantes por variable (Figura 4) muestra una alta incidencia de datos ausentes en varias columnas clave. Destaca eBC_tot con casi el 70 % de sus valores faltantes, seguido por eBC y SO2 con cerca del 50 %. Un segundo grupo de variables afectadas significativamente incluye WS, WD (ambas 26.6 %), PM10 (20.3 %) y las fracciones eBC_ff y eBC_bb (ambas 18.8 %). La considerable proporción de datos faltantes, particularmente en indicadores importantes de calidad del aire y fuentes como eBC_tot, SO2 y PM10, confirma la necesidad de realizar una imputación robusta para análisis posteriores.

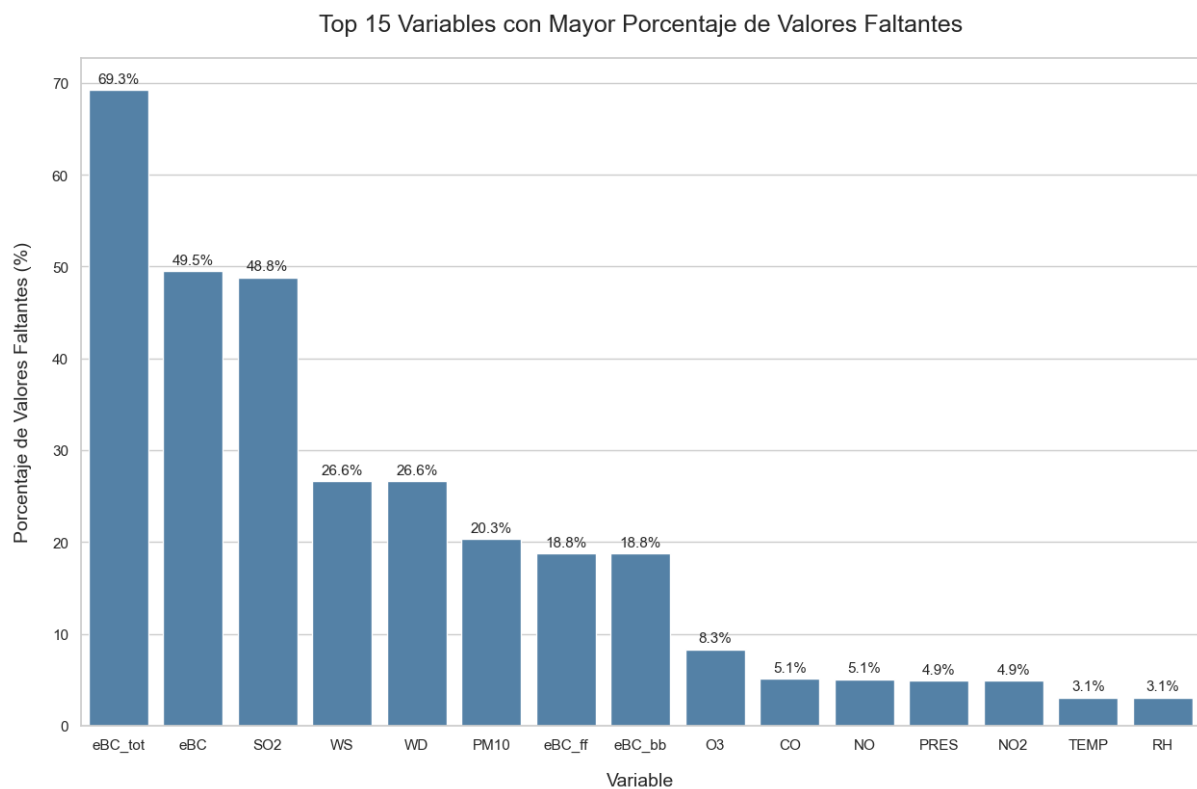


Figura 4: Porcentaje de valores faltantes (nulos) para las 15 variables con mayor proporción de ausencias en el dataset de intersección `trafico_contamina_interseccion`. Se observa la mayor incidencia en eBC_tot (69.3 %), seguida por eBC (49.5 %) y SO2 (48.8 %). Variables como WS, WD, y PM10 también presentan una notable proporción de datos ausentes superior al 20 %.

4.3.2 Distribución de Variables

Se analizaron las distribuciones mediante histogramas y diagramas de caja.

- **Contaminantes (CO, NO, NO₂):** Los histogramas (Figura 5) mostraron distribuciones con fuerte asimetría derecha, indicando predominio de concentraciones bajas con picos ocasionales altos, característico de fuentes como el tráfico.

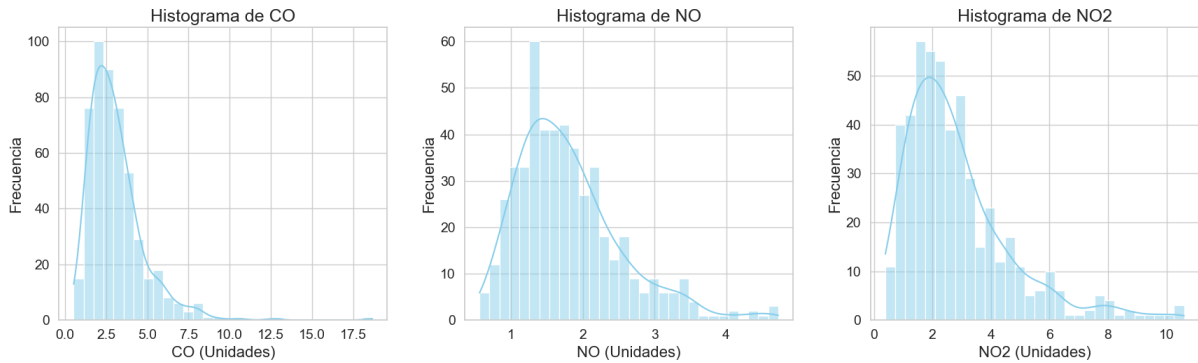


Figura 5: Histogramas de las concentraciones horarias de CO, NO y NO₂. Se observa una clara asimetría hacia la derecha en las tres distribuciones.

- **Contaminantes (O₃, SO₂, PM₁₀, eBC):** Los diagramas de caja (Figura 6 y revelaron valores atípicos, especialmente en O₃ y PM₁₀. La mediana de SO₂ fue muy baja. Las fracciones de eBC mostraron diferencias en sus rangos y medianas.

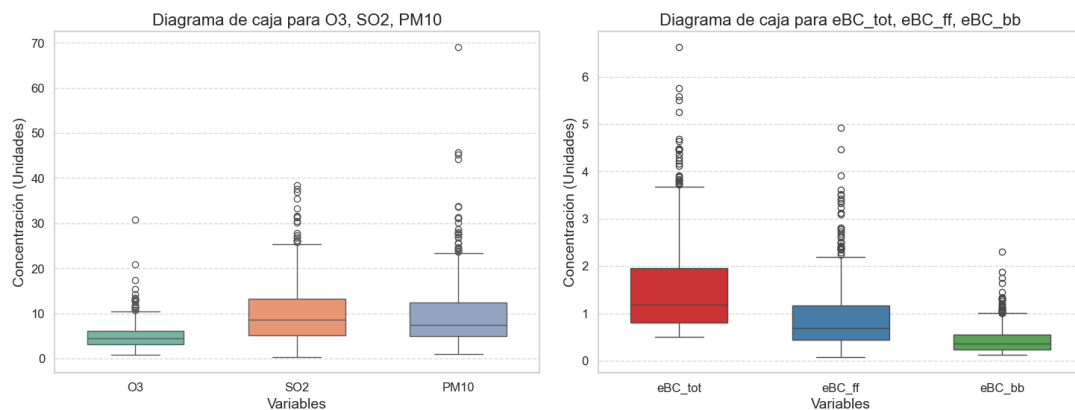


Figura 6: Diagramas de caja comparativos para variables de contaminación. A la izquierda se muestran O₃, SO₂ y PM₁₀. A la derecha se muestran las variables relacionadas con el carbono negro elemental (eBC_tot, eBC_ff, eBC_bb).

- **Meteorología (TEMP, RH, WS):** Los histogramas (Figura 7) mostraron distribuciones más variadas: TEMP bimodal (ciclos día/noche), RH más centrada, y WS con fuerte asimetría derecha (predominio de vientos bajos).

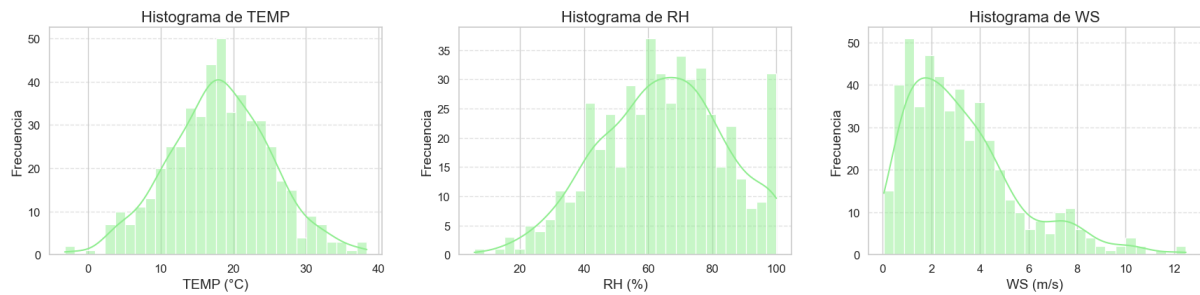


Figura 7: Histogramas de las variables meteorológicas TEMP (Temperatura), RH (Humedad Relativa) y WS (Velocidad del Viento).

4.3.3 Análisis de Correlaciones

La matriz de correlación (Figura 8) mostró relaciones significativas:

- **Positivas Fuertes:** eBC_tot con eBC_ff (0.76) y eBC_bb (0.64); NO2 con eBC_tot (0.69); NO con NO2 (0.64); O3 con TEMP (0.4).
- **Negativas Moderadas:** RH con O3 (-0.5); RH con TEMP (-0.5).
- **Otras:** Correlaciones negativas débiles entre contaminantes primarios (NOx, eBC) y O3. Correlaciones débiles entre meteorología y contaminantes primarios.

Estas correlaciones, especialmente las ligadas a trazadores de tráfico, son clave para la imputación basada en modelos predictivos.

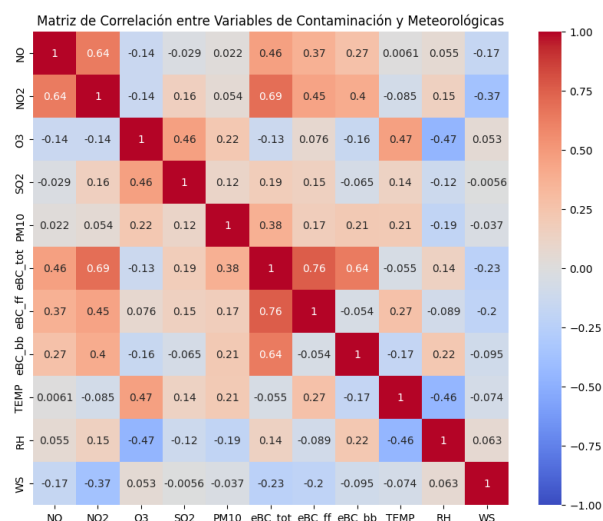


Figura 8: Matriz de correlación de Pearson entre las principales variables de contaminación

4.4 METODOLOGÍA: PIPELINE DE IMPUTACIÓN Y FORMALIZACIÓN

Este capítulo describe la metodología completa empleada para la imputación de valores faltantes. Se detalla la preparación de los datos, la configuración experimental para evaluación, y la aplicación de cada técnica de imputación implementada.

4.4.1 *Desarrollo del Entorno de Imputación*

Para llevar a cabo la parte práctica, centrada en la imputación de datos faltantes en series temporales de contaminación, se ha desarrollado una solución específica mediante la creación de un paquete de Python denominado `pampaneira_imputation`. Este paquete organiza de forma modular toda la lógica necesaria para el preprocesamiento de datos, la aplicación de diversos métodos de imputación y la evaluación de sus resultados.

4.4.1.1 *Estructura del Paquete `pampaneira_imputation`*

El paquete `pampaneira_imputation` ha sido estructurado en varios módulos, cada uno con responsabilidades bien definidas para facilitar el mantenimiento, la escalabilidad y la claridad del código. La estructura principal del paquete es la siguiente:

- **`__init__.py`**: Permite que el directorio sea tratado como un paquete de Python.
- **`config.py`**: Centraliza todos los parámetros de configuración del proyecto, como rutas de archivos, fechas de los periodos, parámetros de los modelos, nombres de columnas, etc., permitiendo una fácil modificación y experimentación.
- **`data_loader.py`**: Contiene las funciones responsables de cargar los conjuntos de datos originales desde sus fuentes (e.g., archivos CSV) y realizar una limpieza o transformación inicial básica.
- **`data_preprocessor.py`**: Implementa la lógica para el preprocesamiento de los datos, incluyendo la división en conjuntos de entrenamiento, validación y prueba, el escalado de características, la creación de ventanas temporales (secuencias) y la introducción artificial de datos faltantes para la evaluación de los modelos de imputación.
- **`evaluation.py`**: Define las funciones para calcular las métricas de rendimiento utilizadas para evaluar la precisión de los métodos de imputación (RMSE, MSE, MAE, MRE).
- **`imputation_methods.py`**: Agrupa las implementaciones de los diferentes algoritmos de imputación evaluados, incluyendo tanto métodos simples (media, mediana, interpolación lineal, `ffill`, `bfill`) como modelos más avanzados basados en deep learning. Parte de la funcionalidad de los modelos avanzados se ha construido aprovechando las capacidades de la librería **PyPOTS** (Du, 2023), una

biblioteca especializada en el procesamiento de datos con patrones irregulares y datos faltantes en series temporales.

- **utils.py**: Recopila funciones auxiliares de propósito general utilizadas a lo largo de todo el paquete, como pueden ser funciones para la creación de ventanas deslizantes, generación de datos faltantes con patrones específicos, o reformato de datos.

4.4.1.2 *Flujo de Trabajo y Experimentación*

El flujo de trabajo principal (pipeline) para la experimentación y la obtención de resultados se ha orquestado y documentado mediante un Jupyter Notebook denominado `Notebook_Imputation.ipynb`. Este notebook guía a través de los siguientes pasos:

1. Carga de la configuración del proyecto.
2. Carga de los datos crudos utilizando el módulo `data_loader`.
3. Preprocesamiento de los datos (división, escalado, ventaneo, introducción de NAs) a través de `data_preprocessor`.
4. Aplicación de los diferentes métodos de imputación definidos en `imputation_methods` sobre los datos preprocesados con valores faltantes.
5. Evaluación del rendimiento de cada método utilizando las métricas de `evaluation` sobre los puntos donde se introdujeron NAs artificialmente.
6. Visualización y análisis de los resultados.

Este enfoque permite una experimentación reproducible y una clara trazabilidad de los resultados obtenidos.

4.4.1.3 *Control de Versiones y Pruebas*

El desarrollo íntegro del código del paquete `pampaneira_imputation` y el Jupyter Notebook se ha gestionado mediante el sistema de control de versiones Git. El repositorio con todo el código fuente y los materiales del proyecto se encuentra disponible públicamente en GitHub¹.

Adicionalmente, para asegurar la correcta funcionalidad de los componentes clave del paquete, se han implementado una serie de **test unitarios**. Estos tests verifican el comportamiento esperado de funciones críticas, especialmente aquellas involucradas en el preprocesamiento de datos y el cálculo de métricas, contribuyendo a la robustez y fiabilidad del software desarrollado.

¹ <https://github.com/eigenric/TFG>

4.5 PREPARACIÓN DE DATOS Y CONFIGURACIÓN EXPERIMENTAL

El proceso se inicia con los datos crudos de series temporales multivariantes de Pampaneira-Bubión.

4.5.1 Carga, Estructura y Preprocesamiento Inicial

Los datos originales, que pueden contener faltantes naturales, son cargados utilizando rutinas del módulo `pampaneira_imputation.data_loader`. Posteriormente, una función del módulo `pampaneira_imputation.data_preprocessor` asegura un índice temporal horario completo y regular para los periodos de interés definidos. Los datos se organizan en una estructura tensorial 3D ($N \times T \times D$), donde N es el número de muestras (ventanas temporales), $T = 24$ es la longitud fija de la ventana, y D el número de características. Los datos se dividen por periodos temporales para análisis separados.

4.5.2 División, Escalado y Creación del Escenario de Evaluación

El núcleo del preprocesamiento, gestionado por el módulo `pampaneira_imputation.data_preprocessor`, aplica los siguientes pasos a los datos de cada periodo:

1. **División Train/Validation/Test:** Segmentación cronológica de los datos en conjuntos de entrenamiento (80 %), validación (20 % del de entrenamiento) y prueba (20 % final), según las duraciones especificadas.
2. **Escalado (Normalización Z-score):** Aplicación de escalado estándar a las características seleccionadas, ajustando el escalador (*fitting*) solo sobre el conjunto de entrenamiento y aplicándolo (*transforming*) a los tres conjuntos.
3. **Ventaneo (Windowing):** Creación de secuencias de longitud $T = 24$ mediante una ventana deslizante, resultando en la estructura tensorial 3D ($N \times T \times D$).
4. **Introducción Artificial de Datos Faltantes:** Se eliminan artificialmente un 10 % de los valores previamente observados (después de escalar y ventanear) en cada split (train, val, test), siguiendo un patrón configurado (ej., 'point'). Este paso es crucial para la evaluación controlada.

Este proceso define formalmente los siguientes conjuntos de datos y máscaras utilizados en el experimento:

- $\mathcal{X}_{\text{orig}}$: El tensor de datos escalados y ventaneados *antes* de introducir los faltantes artificiales. Contiene los faltantes naturales originales y sirve como **ground truth** para la evaluación.

- $\mathbf{M}_{\text{natural}}$: La máscara binaria que indica la posición de los faltantes naturales en $\mathcal{X}_{\text{orig}}$ (0 =faltante, 1 =observado).
- $\mathbf{M}_{\text{artificial}}$: La máscara binaria que indica las posiciones donde se introdujeron faltantes *artificialmente* (1 =eliminado artificialmente, 0 =no eliminado).
- $\mathcal{X}_{\text{missing}}$: El tensor de datos utilizado como **entrada** para los métodos de imputación. Contiene tanto los faltantes naturales como los artificiales.

4.6 PROBLEMA DE IMPUTACIÓN Y OBJETIVO DE EVALUACIÓN

Definición 43 (Problema de Imputación Experimental y Evaluación). El problema abordado es desarrollar y evaluar funciones de imputación f que, aplicadas al conjunto de test con faltantes combinados $(\mathcal{X}_{\text{missing}}^{\text{test}}, \mathbf{M}^{\text{test}})$, generen una estimación completada $\hat{\mathcal{X}}^{\text{test}}$:

$$\hat{\mathcal{X}}^{\text{test}} = f(\mathcal{X}_{\text{missing}}^{\text{test}}, \mathbf{M}^{\text{test}})$$

Para modelos predictivos f_{θ} , la función f depende de parámetros θ^* optimizados durante una fase de entrenamiento previa. El **objetivo principal de evaluación** es medir la precisión con la que $\hat{\mathcal{X}}^{\text{test}}$ reconstruye los valores verdaderos $\mathcal{X}_{\text{orig}}^{\text{test}}$ exclusivamente en las posiciones indicadas por la máscara artificial $\mathbf{M}_{\text{artificial}}^{\text{test}}$ (o $\mathbf{M}_{\text{indicating}}^{\text{test}}$):

$$\text{Error} = \text{Metric}(\hat{\mathcal{X}}^{\text{test}}, \mathcal{X}_{\text{orig}}^{\text{test}}, \mathbf{M}_{\text{artificial}}^{\text{test}})$$

Se busca la función f que minimice este error (medido por MAE, MSE, RMSE, MRE) en el conjunto de test.

4.7 IMPLEMENTACIÓN DE LAS FUNCIONES DE IMPUTACIÓN

Las siguientes subsecciones describen la implementación conceptual de cada función de imputación f , gestionada por el módulo `pampaneira_imputation.imputation_methods`, aplicada sobre $\mathcal{X}_{\text{missing}}^{\text{test}}$.

4.7.1 Métodos Baseline o Estadísticos Simples

Imputación por Media/Mediana Sample-Wise

Procesa cada muestra $\mathbf{X}^{(k)}$ de 24 horas independientemente. Para cada característica, calcula la media/mediana usando solo los valores observados dentro de esa muestra y rellena los NaN de esa característica en esa muestra con el valor calculado. Implementa una lógica de fallback (usar estadística global del training set o 0) si una característica es completamente NaN dentro de una muestra.

4.7.2 Métodos Basados en Propagación e Interpolación

Estos métodos también operan independientemente sobre cada muestra $\mathbf{X}^{(k)}$ utilizando técnicas estándar de series temporales (implementadas internamente con ayuda de bibliotecas como pandas).

Imputación Hacia Adelante (Forward Fill / LOCF)

Propaga la última observación válida hacia adelante en el tiempo ($\text{axis}=0$). Los NaN iniciales o persistentes se rellenan con 0.

Imputación Hacia Atrás (Backward Fill / NOCB)

Propaga la siguiente observación válida hacia atrás en el tiempo ($\text{axis}=0$). Los NaN finales o persistentes se rellenan con 0.

Imputación por Interpolación Lineal

Estima los valores faltantes mediante interpolación lineal entre los puntos observados vecinos más cercanos en el tiempo ($\text{axis}=0$), extrapolando en los extremos si es posible. Los NaN restantes se rellenan con 0.

4.7.3 Métodos Basados en Modelos Predictivos (Deep Learning)

Imputación con Transformer y SAITS

Se utilizan las arquitecturas Transformer y SAITS implementadas en la biblioteca PyPOTS (Du, 2023), cuyas funcionalidades detalladas se pueden consultar en su documentación ². El proceso se divide en:

1. *Entrenamiento*: Una función dentro de `imputation_methods` instancia el modelo PyPOTS seleccionado (SAITS o Transformer) con hiperparámetros específicos. El modelo se entrena sobre los conjuntos de entrenamiento y validación ($\mathcal{X}_{\text{missing}}^{\text{train/val}}, \mathbf{M}^{\text{train/val}}$), utilizando un optimizador estándar (Adam), un planificador de tasa de aprendizaje (ReduceLROnPlateau) y una función de pérdida apropiada (SmoothL1Loss). El resultado es un modelo con parámetros θ^* optimizados.

² <https://docs.pypots.com/en/latest/pypots.imputation.html>

2. *Imputación (Inferencia)*: Otra función del módulo toma el modelo entrenado f_{θ^*} y lo aplica al conjunto de test ($\mathcal{X}_{\text{missing}}^{\text{test}}, \mathbf{M}^{\text{test}}$) para generar la predicción completada $\hat{\mathcal{X}}^{\text{test}}$.

4.8 VISUALIZACIÓN DE SERIES TEMPORALES IMPUTADAS

A continuación, se presentan las series temporales para algunas de las variables de contaminación clave, mostrando los datos originales (ground truth), los puntos donde se introdujo la ausencia de datos artificialmente, y los valores imputados por el método Saits.

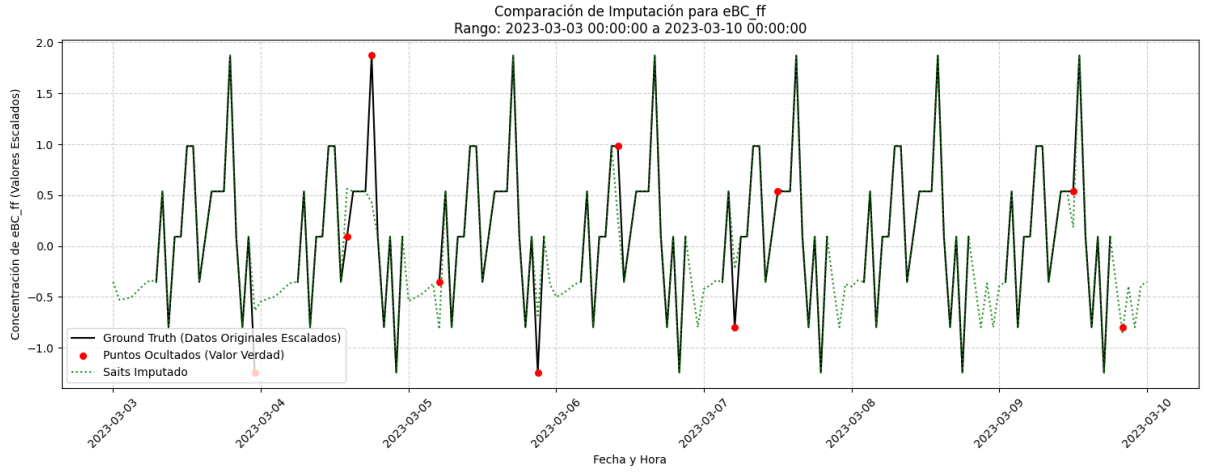


Figura 9: Comparación de imputación para la variable eBC_ff.

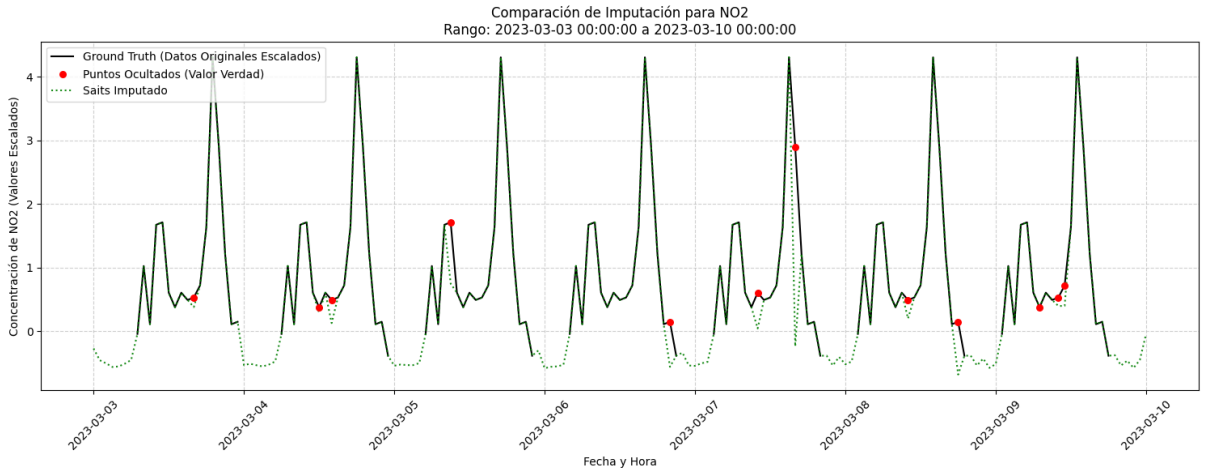


Figura 10: Comparación de imputación para la variable NO2.

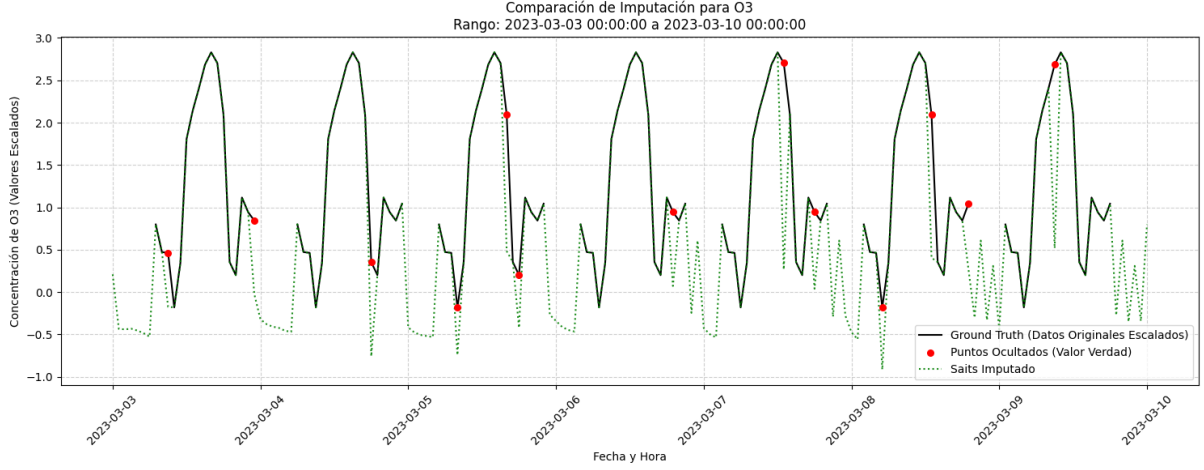


Figura 11: Comparación de imputación para la variable O₃.

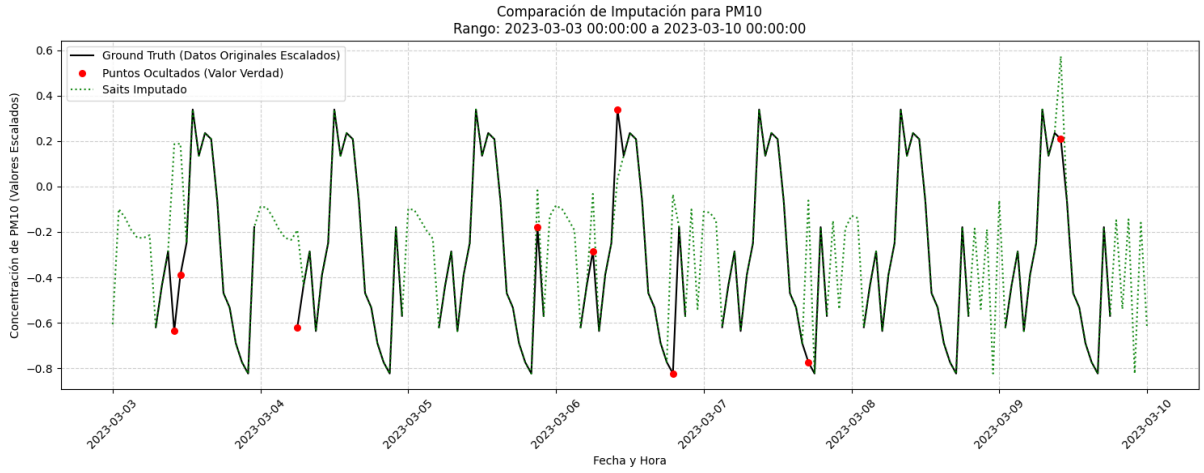


Figura 12: Comparación de imputación para la variable PM₁₀.

En estas figuras, la línea continua negra representa el *ground truth* (datos originales escalados), los puntos rojos indican los valores que fueron ocultos artificialmente para la prueba del modelo, y la línea punteada verde corresponde a los valores imputados por el modelo Saits. Se puede observar cómo el modelo de imputación sigue las tendencias generales de los datos originales y la precisión con la que reconstruye los valores en los puntos ocultos.

4.9 EVALUACIÓN DEL RENDIMIENTO

El rendimiento final de cada método se evalúa utilizando el módulo `pampaneira_imputation.evaluation`. Este calcula las métricas MAE, MSE, RMSE y MRE comparando las predicciones $\hat{\mathcal{X}}^{\text{test}}$ con el ground truth $\mathcal{X}_{\text{orig}}^{\text{test}}$ únicamente en las posiciones indicadas por la máscara artificial $\mathbf{M}_{\text{artificial}}^{\text{test}}$.

Las siguientes tablas presentan los resultados de la evaluación de siete métodos de imputación de datos (Median, Mean, Linear, Ffill, Bfill, Transformer, Saits) en dos periodos distintos, utilizando cuatro métricas de error: RMSE (Error Cuadrático Medio Raíz), MSE (Error Cuadrático Medio), MAE (Error Absoluto Medio) y MRE (Error Relativo Medio).

Cuadro 1: Resultados de Evaluación - Periodo 1

Métrica	Median	Mean	Linear	Ffill	Bfill	Transformer	Saits
RMSE	1,1342	1,0521	0,8796	1,0604	1,0487	0,6471	0,6370
MSE	1,2865	1,1069	0,7737	1,1244	1,0997	0,4187	0,4057
MAE	0,6910	0,7346	0,4621	0,5618	0,5554	0,3378	0,3291
MRE	0,9311	0,9898	0,6226	0,7569	0,7484	0,4552	0,4434

Cuadro 2: Resultados de Evaluación - Periodo 2

Métrica	Median	Mean	Linear	Ffill	Bfill	Transformer	Saits
RMSE	1,1103	1,0609	0,9252	1,0754	1,1018	0,7361	0,7345
MSE	1,2329	1,1255	0,8559	1,1565	1,2140	0,5419	0,5395
MAE	0,7177	0,7637	0,4915	0,5836	0,5894	0,4048	0,3962
MRE	0,9161	0,9747	0,6273	0,7470	0,7544	0,5167	0,5057

INTERPRETACIÓN DE LOS RESULTADOS

Para todas estas métricas, **valores más bajos indican un mejor rendimiento** del método de imputación, ya que representan un menor error entre los valores imputados y los valores reales.

Discusión General del Rendimiento de los Métodos

- **Dominio de Modelos Avanzados:** Los modelos basados en arquitecturas Transformer, específicamente **Saits** y **Transformer**, demostraron consistentemente una precisión superior en la imputación de datos faltantes en comparación con todos los demás métodos. Esto se reflejó en errores notablemente inferiores en todas las métricas y en ambos periodos de estudio.
- **Liderazgo de Saits:** Dentro del grupo de modelos avanzados, **Saits** se destacó por ofrecer el rendimiento más preciso de manera generalizada, obteniendo los

errores más bajos en la mayoría de las evaluaciones, aunque seguido de cerca por el modelo **Transformer**.

■ **Eficacia de Métodos Tradicionales y Simples:**

- Entre los enfoques más convencionales, la imputación **Linear** (interpolación lineal) se posicionó como la más efectiva, superando consistentemente a los métodos basados en la mediana, la media, y las técnicas de propagación de valores (forward fill y backward fill).
- El rendimiento de los métodos **Ffill** y **Bfill** fue inconsistente entre sí y en comparación con la imputación por media (**Mean**) y mediana (**Median**), variando su efectividad relativa entre los periodos analizados.
- Por lo general, los métodos más simples, como la imputación por **Mean** y **Median**, tendieron a registrar los errores más elevados, indicando una menor capacidad para reconstruir con precisión los datos faltantes.

Análisis Comparativo entre Periodos de Estudio

- **Variación del Error entre Periodos:** Se observó una tendencia general hacia errores de imputación ligeramente mayores durante el Periodo 2 en comparación con el Periodo 1 para la mayoría de los métodos, especialmente los de mejor rendimiento. Esta diferencia podría deberse a que el Periodo 2 es mucho más reducido.
- **Estabilidad en la Jerarquía de Métodos:** A pesar de las fluctuaciones en los niveles de error absoluto entre los dos periodos, la jerarquía de rendimiento relativo entre los modelos se mantuvo en gran medida estable. Los modelos **Saits** y **Transformer** continuaron siendo los más precisos, seguidos por la imputación **Linear**.

Observaciones Clave por Tipo de Métrica de Error

- **Impacto de Errores Grandes (RMSE y MSE):** Las métricas que magnifican el impacto de errores grandes, como RMSE y MSE, confirmaron la robustez de **Saits** y **Transformer**. Sus bajos valores en estas métricas sugieren que estos modelos no solo son precisos en promedio, sino que también son menos propensos a cometer errores de gran magnitud.
- **Error Promedio Absoluto (MAE):** El MAE, que mide la desviación promedio absoluta, también resaltó la superioridad de **Saits** y **Transformer**, indicando que sus imputaciones se encuentran, en promedio, más próximas a los valores verdaderos.
- **Error Relativo (MRE):** Al considerar el error desde una perspectiva porcentual a través del MRE, se reafirmó la mayor precisión de los modelos **Saits** y **Transformer** en comparación con los otros métodos evaluados.

W

Parte IV

BLOQUE IV: CONCLUSIONES Y TRABAJO FUTURO

CONCLUSIONES Y TRABAJO FUTURO

Los resultados demuestran claramente que los modelos basados en arquitecturas Transformer, y en particular **Saits**, ofrecen una capacidad de imputación de datos faltantes sustancialmente mejor que los métodos tradicionales y la interpolación lineal simple. Si la precisión es un factor crítico, la inversión en la implementación de estos modelos más complejos está justificada. Entre los métodos más simples, la interpolación lineal (**Linear**) es la opción preferible. La ligera degradación del rendimiento en el Periodo 2 para todos los métodos sugiere que las características de los datos pueden influir en la dificultad de la tarea de imputación.

5.1 TRABAJO FUTURO

5.1.1 *Mejora de los Modelos Transformer*

5.1.2 *Extensiones a Otros Sistemas IoT*

5.1.3 *Posibles Aplicaciones en Diferentes Dominios*

BIBLIOGRAFÍA

- Abu-Mostafa, Y. S., Magdon-Ismael, M., and Lin, H.-T. (2012). *Learning from Data*. AMLBook.
- Apostol, T. M. (1974). *Mathematical Analysis*. Addison-Wesley, 2nd edition.
- Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Burden, R. L. and Faires, J. D. (2011). *Numerical Analysis*. Brooks/Cole, 9th edition.
- Casella, G. and Berger, R. L. (2002). *Statistical Inference*. Duxbury Press, 2nd edition.
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314.
- De, S., Bermudez-Edo, M., Xu, H., and Cai, Z. (2022). Deep Generative Models in the Industrial Internet of Things: A Survey. *IEEE Transactions on Industrial Informatics*, 18(9):5728–5737.
- DeGroot, M. H. and Schervish, M. J. (2012). *Probability and Statistics*. Addison-Wesley, 4th edition.
- Du, W. (2023). Pypots: a python toolbox for data mining on partially-observed time series. *arXiv preprint arXiv:2305.18811*.
- Golub, G. H. and Van Loan, C. F. (2013). *Matrix Computations*. Johns Hopkins University Press, 4th edition.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.
- Kincaid, D. R. and Cheney, E. W. (2002). *Numerical Analysis: Mathematics of Scientific Computing*. Brooks/Cole, 3rd edition.
- Kolda, T. G. and Bader, B. W. (2009). Tensor decompositions and applications. *SIAM Review*, 51(3):455–500.
- Kreyszig, E. (1989). *Introductory Functional Analysis with Applications*. Wiley.
- Noble, B. and Daniel, J. W. (1988). *Applied Linear Algebra*. Prentice Hall, 3rd edition.

- Rohatgi, V. K. and Saleh, A. K. M. E. (2015). *An Introduction to Probability and Statistics*. Wiley, 3rd edition.
- Ross, S. M. (2014). *A First Course in Probability*. Pearson, 9th edition.
- Rudin, W. (1976). *Principles of Mathematical Analysis*. McGraw-Hill, 3rd edition.
- Rudin, W. (1987). *Real and Complex Analysis*. McGraw-Hill, 3rd edition.
- Rudin, W. (1991). *Functional Analysis*. McGraw-Hill, 2nd edition.
- Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088):533–536.
- Sandradewi, J., Prévôt, A. S. H., Szidat, S., Perron, N., Alfarra, M. R., Lanz, V. A., Weingartner, E., and Baltensperger, U. (2008). Source apportionment of carbonaceous aerosol using the Aethalometer model. *Atmospheric Chemistry and Physics*, 8(11):2749–2760.
- Strang, G. (2016). *Introduction to Linear Algebra*. Wellesley-Cambridge Press, 5th edition.
- Universidad de Granada (2023). Smartpoqueira: Experimentación de tecnologías inteligentes para la gestión sostenible de las visitas a zonas rurales y parques nacionales. Proyecto de investigación.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30.

ANEXOS

PLANIFICACIÓN DEL TRABAJO Y ANÁLISIS DEL PRESUPUESTO

En esta sección se detalla tanto la planificación estimada inicial como la seguida durante el desarrollo del Trabajo Fin de Grado (TFG), junto con un análisis del presupuesto basado en el tiempo real invertido. La planificación y el seguimiento se visualizan a través de diagramas de Gantt generados con el software de código abierto **GanttProject**¹. El análisis presupuestario se fundamenta en las **horas reales de trabajo registradas** mediante la herramienta de gestión de tiempo de código abierto **Timewarrior**². El objetivo es ofrecer una valoración transparente y fiel a la dedicación invertida en el proyecto, contrastando la planificación inicial con la ejecución real.

Planificación del Trabajo: Estimación vs. Realidad

El diseño inicial y el posterior seguimiento de la planificación del proyecto se han gestionado utilizando GanttProject. En el **Anexo A** se presentan dos diagramas de Gantt para ilustrar la diferencia entre la estimación inicial y el progreso real:

- La **Figura 13** muestra una planificación inicial optimista, elaborada al comienzo del proyecto.
- La **Figura 14** refleja la progresión y duración real de las tareas, tal como se desarrollaron.

¹ <https://www.ganttproject.biz/>

² <https://timewarrior.net/>

ANEXO A: CRONOGRAMAS

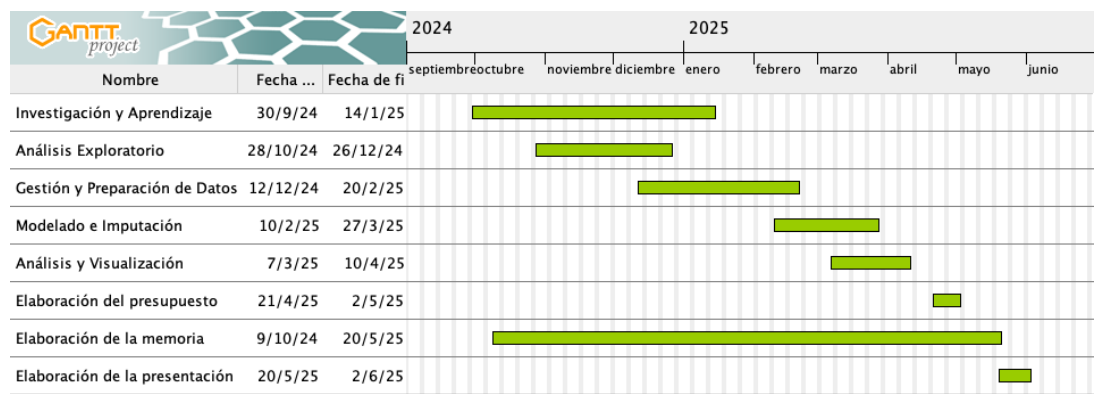


Figura 13: Diagrama de Gantt de la planificación inicial estimada para la realización del TFG.



Figura 14: Diagrama de Gantt del progreso real durante la realización del TFG.

La comparación entre ambos cronogramas evidencia que el desarrollo real del proyecto sufrió una desviación con respecto a la planificación inicial. Este retraso se debió principalmente a los siguientes factores:

- **Profundización en el Estado del Arte:** La revisión bibliográfica y el estudio de trabajos previos requirieron más tiempo del previsto para establecer una base teórica robusta y contextualizar adecuadamente el proyecto.
- **Complejidad de la Arquitectura Transformer:** Comprender en detalle los mecanismos de atención y la implementación de los modelos Transformer supuso una inversión de tiempo considerable, fundamental para su correcta aplicación.
- **Curva de Aprendizaje de Bibliotecas:** La familiarización y el manejo eficiente de las bibliotecas de Python utilizadas (e.g., PyPOTS basada en PyTorch, Pandas, Numpy para manipulación de datos y Matplotlib para visualización) implicaron un periodo de adaptación necesario para la implementación práctica.

Análisis del Presupuesto del Proyecto

El presupuesto presentado a continuación se basa en la contabilización precisa de las horas dedicadas a cada tarea del TFG, registradas sistemáticamente con Timewarrior a lo largo de todo el desarrollo. Para la valoración económica, se ha aplicado una tarifa de 10 euros/hora.

Para facilitar la comprensión y el análisis del presupuesto, el trabajo registrado se ha clasificado en tres categorías principales, siguiendo una estructura habitual:

- **Parte teórica:** Recoge las tareas relacionadas con el análisis conceptual, el estudio del estado del arte y la fundamentación teórica del proyecto. Corresponde a la categoría agrupada *Investigación y Aprendizaje*.
- **Parte práctica:** Reúne las actividades directamente vinculadas con la implementación, programación, experimentación, análisis de datos y validación. Engloba las categorías de *Pipeline y Datos*, *Modelado y Entrenamiento*, *Imputación y Nulos*, *Análisis y Visualización*, y *Metodología y Estructura*.
- **Parte general:** Agrupa las labores transversales de gestión del proyecto, comunicación (reuniones con tutores, correos), documentación y elaboración de la memoria. Corresponde a la categoría *Gestión y Comunicaciones*.

Además de los costes derivados de las horas de trabajo, se incluye el coste amortizado del equipamiento principal utilizado (Macbook Air M2), considerando una vida útil estimada de 7 años. El desglose detallado del presupuesto se presenta en el **Anexo B** (Cuadro 3).

ANEXO B: PRESUPUESTO ESTIMADO

Cuadro 3: Estimación del coste del proyecto basado en horas reales registradas.

Concepto	Tiempo (horas)	Precio (euros/hora)	Coste (euros)
Parte general			
Gestión y Comunicaciones	1,4	10	14,00
Parte teórica			
Investigación y Aprendizaje	278,8	10	2788,00
Parte práctica			
Pipeline y Datos	43,6	10	436,00
Modelado y Entrenamiento	40,6	10	406,00
Imputación y Nulos	230,5	10	2305,00
Análisis y Visualización	7,0	10	70,00
Metodología y Estructura	4,4	10	44,00
Equipo/Vida útil - Macbook Air M2 / 7 años	-	-	162,70
Coste total del proyecto	606.3	-	6225.70

*Nota: Las horas reflejan el tiempo total registrado con Timewarrior para las tareas agrupadas en cada categoría.
El coste del equipo representa una amortización estimada durante el periodo del proyecto.*